

```
1 // ===== FILE: Assets/Scripts/Core/GameBootstrap.cs =====
2 using UnityEngine;
3 namespace InkLine
4 {
5     public static class GameBootstrap
6     {
7         [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterSceneLoad)]
8         static void Boot()
9         {
10             Application.targetFrameRate = 60;
11             Input.simulateMouseWithTouches = true;
12             if (Object.FindObjectOfType<GameFlow>() != null) return;
13             var root = new GameObject("InkLine");
14             Object.DontDestroyOnLoad(root);
15             root.AddComponent<GameFlow>();
16         }
17     }
18 }
19 // ===== FILE: Assets/Scripts/Core/GameFlow.cs =====
20 using System.Collections.Generic;
21 using System.IO;
22 using UnityEngine;
23 using UnityEngine.EventSystems;
24 using UnityEngine.UI;
25 namespace InkLine
26 {
27     public sealed class GameFlow : MonoBehaviour
28     {
29         enum Screen { Lobby, Intro, Battle, Draft, Place, Confirm, Result }
30         MetaProgress _meta;
31         BattleWorld _world;
32         BattleView _view;
33         BattleHud _hud;
34         HomeScreen _home;
35         Canvas _canvas;
36         RectTransform _layer;
37         Screen _screen = Screen.Lobby;
38         int _pickStage;
39         ResultInfo _result;
40         bool _inkDoubled;
41         CardId[] _offer = new CardId[3];
42         CardId _held;
43         bool _rerolled;
44         int _draftPaid;
45         bool _taughtStar;
46         bool _dragging;
47         // 金币够了自动弹改装。够了先等一小下再弹，免得刚捡到钱画面就被盖住；
48         // 玩家自己关掉改装框，说明这会儿想攒钱，静默一阵再弹。
49         float _autoWait;
50         float _autoMute;
```

```
51     const float AutoDelay = 0.35f;
52     const float AutoMuteAfterClose = 6f;
53     string _tip;
54     Transform _overlay;
55     int _shotPage = -1;
56     readonly bool[] _shotGot = new bool[4];
57     readonly Queue<string> _codexToasts = new Queue<string>();
58     float _audioSweep;
59     void Start()
60     {
61         Analytics.Ensure();
62         WxBridge.InitSdk() =>
63         {
64             AdStub.Warm();
65             CloudSync.Startup(Begin);
66         });
67         CloudSync.Imported += OnCloudImported;
68     }
69     void OnDestroy()
70     {
71         CloudSync.Imported -= OnCloudImported;
72     }
73     // 进大厅后才到的云端档（启动超时后晚到、或上行被 409 打回）。
74     // 战斗里只换存档不打断，结算时就记到新档上。
75     void OnCloudImported()
76     {
77         if (_canvas == null) return;
78         _meta = MetaProgress.Load();
79         if (_screen == Screen.Lobby) ShowHome();
80     }
81     void Begin()
82     {
83         if (_canvas != null) return;
84         _meta = MetaProgress.Load();
85         _canvas = UIKit.CreateCanvas("InkUI");
86         _layer = _canvas.GetComponent<RectTransform>();
87         GmBar.Preview = GmPreview;
88         ScreenFit.Apply(Camera.main, _canvas);
89         string force = Path.Combine(Application.dataPath, "../Library/ink_shot_preview.force");
90         if (File.Exists(force))
91             BattleWorld.PreviewFill = true;
92         ShowHome();
93         if (BattleWorld.PreviewFill) StartStage(0);
94     }
95     void Update()
96     {
97         if (_canvas == null) return;
98         ScreenFit.Apply(Camera.main, _canvas);
99         AudioBus.Duck(_screen == Screen.Draft || _screen == Screen.Confirm || _screen == Scre...
100        AudioBus.Tick();
```

```
101     _audioSweep -= Time.unscaledDeltaTime;
102     if (_audioSweep <= 0f)
103     {
104         _audioSweep = 0.35f;
105         AudioBus.Sweep(_layer);
106     }
107     InkPointer.Pump();
108     bool uiHit = EventSystemOverUi();
109     if (_screen == Screen.Lobby && _home != null) _home.Tick();
110     if (_screen == Screen.Intro || _screen == Screen.Battle || _screen == Screen.Place || ...
111     {
112         if (_world != null && _hud != null)
113         {
114             // 掉落物要飞进顶栏，得先知道两个药丸现在在世界的哪儿 --
115             // 安全区一变（转屏、不同机型）位置就不一样，不能写死。
116             _world.GoldChip = BattleHud.ChipInWorld(_hud.GoldChip, _world.GoldChip);
117             _world.InkChip = BattleHud.ChipInWorld(_hud.InkChip, _world.InkChip);
118         }
119         if (_world != null && _screen == Screen.Battle)
120         {
121             if (!uiHit) HandleRail();
122             _world.Tick(Time.deltaTime);
123             PumpCodexToast();
124             TickAutoDraft();
125         }
126         if (_world != null && _view != null)
127         {
128             _view.Sync(_world);
129             if (_screen == Screen.Place) PaintPlaceHighlights();
130         }
131         // 先把最后一帧画出来再结算。原先胜负一成立就 return，
132         // 死亡当帧的画面被跳过，怪还站在场上通关页就盖上来。
133         if (_world != null && _screen == Screen.Battle && !_askingRetreat)
134         {
135             if (_world.Victory) { Finish(true); return; }
136             if (_world.Defeat) { Finish(false); return; }
137         }
138         if (_hud != null)
139         {
140             if (BattleWorld.PreviewFill && _world != null && !string.IsNullOrEmpty(_world...
141                 _tip = _world.PreviewLabel;
142             _hud.Refresh(_world, _screen == Screen.Battle, _tip);
143         }
144         if (BattleWorld.PreviewFill && _world != null && _screen == Screen.Battle)
145             TryCapturePreview();
146         if (!uiHit && _screen == Screen.Place) HandlePlaceClick();
147     }
148 }
149 void ReloadSave()
150 {
```

```

151     _meta = MetaProgress.Load();
152     ShowHome();
153 }
154 void ShowHome() => ShowHome(HomeScreen.TabSortie);
155 bool _askingRetreat;
156 // 体力进关就扣了。撤退前说清楚：进度、没奖励、体力不退。
157 void AskRetreat()
158 {
159     if (_world == null || _askingRetreat) return;
160     if (_screen != Screen.Battle && _screen != Screen.Intro) return;
161     _askingRetreat = true;
162     bool wasPaused = _world.Paused;
163     _world.Paused = true;
164     int pct = Mathf.Clamp(Mathf.RoundToInt(_world.Progress * 100f), 0, 99);
165     var dim = UIKit.Dimmer(_layer);
166     dim.name = "retreat_ask";
167     void Stay()
168     {
169         if (!_askingRetreat) return;
170         AudioBus.Tap();
171         _askingRetreat = false;
172         if (_world != null) _world.Paused = wasPaused;
173         if (dim != null) Destroy(dim.gameObject);
174     }
175     void Leave()
176     {
177         if (!_askingRetreat) return;
178         AudioBus.Tap();
179         _askingRetreat = false;
180         if (dim != null) Destroy(dim.gameObject);
181         Analytics.LevelFail("give_up", pct);
182         ShowHome();
183     }
184     var board = PanelKit.Board(dim, "确认撤退", Vector2.zero, new Vector2(560f, 460f), Pin.Ce...
185     var line = UIKit.Label(board, "pct", $"已完成 {pct}%", 36, new Vector2(0f, 150f),
186         new Vector2(480f, 52f), TextAnchor.MiddleCenter, Pin.Top);
187     UIKit.Bold(line);
188     var note = UIKit.Label(board, "note", "撤退后没有奖励，体力也不退还", 26, new Vector2(0f, 214f),
189         new Vector2(480f, 40f), TextAnchor.MiddleCenter, Pin.Top);
190     note.color = InkTheme.TextMid;
191     UIKit.Btn(board, "stay", "继续战斗", new Vector2(0f, 126f), new Vector2(400f, 92f), Stay,...
192     UIKit.Btn(board, "leave", "确认撤退", new Vector2(0f, 28f), new Vector2(400f, 80f), Leave...
193 }
194 void ShowHome(int tab)
195 {
196     _meta.StorePending();
197     _screen = Screen.Lobby;
198     ClearLayer();
199     if (_view != null) { _view.Dispose(); _view = null; }
200     _world = null;

```

```
201     _hud = null;
202     BattleHud.ReleaseCamera();
203     _home = HomeScreen.Build(_layer, _meta, StartStage, ReloadSave, tab);
204     AudioBus.Music("bgm_home");
205     AudioBus.Warm("bgm_battle");
206 }
207 void StartStage(int index)
208 {
209     _meta.StorePending();
210     if (!BattleWorld.PreviewFill)
211     {
212         int cost = _meta.StageCost(index);
213         if (!_meta.CanEnter(index)) return;
214         _meta.SpendStamina(cost);
215     }
216     _home = null;
217     _pickStage = index;
218     _taughtStar = false;
219     _tip = index == 0 ? "滑到底下那一串，对准敌人。" : "";
220     _world = new BattleWorld();
221     _world.ItemRanks = _meta.ItemLevel;
222     _world.Begin(StageCatalog.Get(index), _meta.Forged, _meta.Equipped);
223     _world.ApplySkin(_meta.Skin);
224     _world.CodexHit = OnCodex;
225     _codexToasts.Clear();
226     if (_view != null) _view.Dispose();
227     _view = new BattleView(null);
228     _view.SetBackdrop(_world.Stage.Chapter);
229     _view.EmitterSkin = _meta.Skin;
230     _view.EmitterTint = _meta.SkinTint;
231     if (!BattleWorld.PreviewFill)
232         Analytics.LevelStart(index + 1, _world.Stage != null ? _world.Stage.Name : "");
233     BuildBattleHud();
234     _autoWait = 0f;
235     _autoMute = 0f;
236     AudioBus.Music("bgm_battle");
237     if (BattleWorld.PreviewFill)
238     {
239         _screen = Screen.Battle;
240         return;
241     }
242     _screen = Screen.Intro;
243     BattleBanner.Show(_layer, _world.Stage, () =>
244     {
245         if (_world == null || _screen != Screen.Intro) return;
246         _screen = Screen.Battle;
247     });
248 }
249 void TickAutoDraft()
250 {
```

```

251     if (_askingRetreat || _world == null || BattleWorld.PreviewFill || _world.Victory || ...
252     float dt = Time.unscaledDeltaTime;
253     if (_autoMute > 0f) _autoMute -= dt;
254     if (!_world.CanDraft || _autoMute > 0f)
255     {
256         _autoWait = 0f;
257         return;
258     }
259     // 正按着滑炮台的时候不弹, 否则松手那一下会点到牌上。
260     if (_dragging || InkPointer.Held) return;
261     _autoWait += dt;
262     if (_autoWait < AutoDelay) return;
263     _autoWait = 0f;
264     OpenDraft();
265 }
266 void OnCodex(CodexKind kind, int index)
267 {
268     if (_world == null || !_meta.CodexLearn(kind, index)) return;
269     string head = kind == CodexKind.Pair ? "秘卷现世"
270         : kind == CodexKind.Enemy ? "墨谱新页" : "图鉴收录";
271     _codexToasts.Enqueue($"{head} · {CodexCatalog.Title(kind, index)}");
272 }
273 // 和关卡规则、成词提示共用一条 toast, 排队等上一条放完, 免得互相顶掉。
274 void PumpCodexToast()
275 {
276     if (_codexToasts.Count == 0 || _world.ToastTime > 0f) return;
277     _world.ShowToast(_codexToasts.Dequeue(), 1.8f);
278 }
279 void BuildBattleHud()
280 {
281     ClearLayer();
282     _hud = BattleHud.Build(_layer, _world, AskRetreat, () =>
283     {
284         if (_screen != Screen.Battle || !_world.CanDraft) return;
285         OpenDraft();
286     }, () =>
287     {
288         if (_screen != Screen.Battle || _world == null) return;
289         if (_world.EmitterCount >= GameConstants.AdEmitterCap) return;
290         AdStub.Reward("emitter", () =>
291         {
292             if (_world != null) _world.AddEmitter();
293         });
294     });
295     _hud.GunSkin = _meta.Skin;
296 }
297 static bool EventSystemOverUi()
298 {
299     if (EventSystem.current == null || !InkPointer.Down && !InkPointer.Held) return false;
300     var ped = new PointerEventData(EventSystem.current) { position = InkPointer.ScreenPos };

```

```
301     var hits = new List<RaycastResult>();
302     EventSystem.current.RaycastAll(ped, hits);
303     for (int i = 0; i < hits.Count; i++)
304     {
305         var g = hits[i].gameObject.GetComponent<Graphic>();
306         if (g != null && g.RaycastTarget) return true;
307         if (hits[i].gameObject.GetComponentInParent<Button>() != null) return true;
308     }
309     return false;
310 }
311 void HandleRail()
312 {
313     if (InkPointer.Down && InRailZone(InkPointer.WorldOnPlane()))
314         _dragging = true;
315     if (_dragging && InkPointer.Held)
316         _world.SetRailFromWorldX(InkPointer.WorldOnPlane().x, false);
317     if (_dragging && InkPointer.Up)
318     {
319         _world.SetRailFromWorldX(InkPointer.WorldOnPlane().x, true);
320         _dragging = false;
321     }
322 }
323 static bool InRailZone(Vector3 world)
324 {
325     return world.y < GameConstants.LeakY - 0.15f;
326 }
327 void OpenDraft()
328 {
329     if (!_world.CanDraft) return;
330     _draftPaid = _world.DraftCost;
331     _world.Gold -= _draftPaid;
332     _world.DraftCount++;
333     _world.Paused = true;
334     _rerolled = false;
335     RollOffer();
336     _screen = Screen.Draft;
337     ShowDraftPanel();
338     AudioBus.Draft();
339 }
340 // 牌池不足三个字时就少发几张。第一关只有「分」一个字，发三张一样的看着像
341 // 出了 bug。顺带修掉原来那句 `pool.Count > 3`：池子正好三个时它不去重，
342 // 会发出重复项。
343 void RollOffer()
344 {
345     List<CardId> pool = new List<CardId>(_world.Stage.Pool);
346     var weight = new List<int>();
347     for (int i = 0; i < pool.Count; i++)
348         weight.Add(OfferWeight(pool[i]));
349     int n = Mathf.Min(3, pool.Count);
350     if (_offer == null || _offer.Length != n) _offer = new CardId[n];
```

```
351     for (int i = 0; i < n; i++)
352     {
353         int pick = WeightedIndex(pool, weight);
354         _offer[i] = pool[pick];
355         pool.RemoveAt(pick);
356         weight.RemoveAt(pick);
357     }
358 }
359 int OfferWeight(CardId id)
360 {
361     CardDef def = CardCatalog.Get(id);
362     if (def.Wake != CardWake.WordPart) return 10;
363     CardId mate = CardCatalog.Partner(id);
364     if (mate != CardId.None && HasSameOnBoard(mate)) return 22;
365     return 6;
366 }
367 static int WeightedIndex(List<CardId> pool, List<int> weight)
368 {
369     int sum = 0;
370     for (int i = 0; i < weight.Count; i++) sum += weight[i];
371     int roll = Random.Range(0, Mathf.Max(1, sum));
372     for (int i = 0; i < weight.Count; i++)
373     {
374         roll -= weight[i];
375         if (roll < 0) return i;
376     }
377     return pool.Count - 1;
378 }
379 void ShowDraftPanel()
380 {
381     DropOverlay();
382     _overlay = DraftPanel.Show(_layer, _tip, _offer, _rerolled, Pick, () =>
383     {
384         AdStub.Reward("reroll", () =>
385         {
386             _rerolled = true;
387             RollOffer();
388             ShowDraftPanel();
389         });
390     }, CloseDraft);
391 }
392 void CloseDraft()
393 {
394     if (_world == null || _screen != Screen.Draft) return;
395     _world.Gold += _draftPaid;
396     _world.DraftCount = Mathf.Max(0, _world.DraftCount - 1);
397     _autoMute = AutoMuteAfterClose;
398     _tip = "";
399     ResumeBattle();
400     AudioBus.Back();
```

```
401     }
402     void Pick(CardId id)
403     {
404         _held = id;
405         if (_world.Stage.TeachStar && !_taughtStar && HasSameOnBoard(id))
406         {
407             _taughtStar = true;
408             _tip = "点空格铺开，或点已有同牌升到 2 星。";
409         }
410         else _tip = "点一个格子放下。";
411         _screen = Screen.Place;
412         DropOverlay();
413     }
414     bool HasSameOnBoard(CardId id)
415     {
416         for (int c = 0; c < GameConstants.Columns; c++)
417             for (int r = 0; r < _world.OpenRows; r++)
418                 if (_world.Grid[c, r] == id) return true;
419         return false;
420     }
421     void HandlePlaceClick()
422     {
423         if (!InkPointer.Down || Camera.main == null) return;
424         Vector3 w = InkPointer.WorldOnPlane();
425         if (!FieldLayout.TryCellAt(w, _world.OpenRows, out int col, out int row)) return;
426         var result = _world.PeekPlace(_held, col, row);
427         // 同一排里没开的格子也点得到（TryCellAt 只按行卡范围），得在这儿拦下来，
428         // 否则会悄悄放进一个画成灰底的格子里。
429         if (result == BattleWorld.PlaceResult.LockedRow)
430         {
431             _world.DenyCell(col, row);
432             _world.ShowToast("这格还没开");
433             return;
434         }
435         if (result == BattleWorld.PlaceResult.RejectedMaxStar)
436         {
437             _world.DenyCell(col, row);
438             _world.ShowToast("已满星");
439             return;
440         }
441         if (result == BattleWorld.PlaceResult.NeedConfirm)
442         {
443             _screen = Screen.Confirm;
444             ShowConfirm(col, row);
445             return;
446         }
447         _world.Place(_held, col, row, false);
448         _tip = "";
449         ResumeBattle();
450     }
```

```
451 void ShowConfirm(int col, int row)
452 {
453     DropOverlay();
454     _overlay = ConfirmPanel.Show(_layer, () =>
455     {
456         _world.Place(_held, col, row, true);
457         _tip = "";
458         ResumeBattle();
459     }, () =>
460     {
461         _screen = Screen.Place;
462         DropOverlay();
463     });
464 }
465 void PaintPlaceHighlights()
466 {
467     for (int c = 0; c < GameConstants.Columns; c++)
468     for (int r = 0; r < GameConstants.Rows; r++)
469     {
470         // 没开的格子不画。能放的格子只留黑框，这里不再铺绿色。
471         if (!_world.IsOpen(c, r)) continue;
472         var p = _world.PeekPlace(_held, c, r);
473         Color col = InkTheme.PlaceBad;
474         if (p == BattleWorld.PlaceResult.Placed) col = InkTheme.PlaceOk;
475         else if (p == BattleWorld.PlaceResult.Upgrade) col = InkTheme.PlaceUp;
476         else if (p == BattleWorld.PlaceResult.RejectedMaxStar) col = new Color(0, 0, 0, 0.5);
477         _view.HighlightCell(c, r, col);
478     }
479 }
480 void ResumeBattle()
481 {
482     DropOverlay();
483     _world.Paused = false;
484     _screen = Screen.Battle;
485 }
486 // 结算只算一次。翻倍广告只让面板把墨数再滚一遍，不要再回 Finish，
487 // 否则 ApplyResult 会把这一局拾到的墨再入一次账。
488 // 输了先给续命页；没得续、不续、续了又死，才放失败动画和失败页。
489 void Finish(bool win)
490 {
491     _screen = Screen.Result;
492     _world.Paused = true;
493     _inkDoubled = false;
494     if (win)
495     {
496         AudioBus.Win();
497         Analytics.LevelClear(_world.StarsEarned);
498         _result = _meta.ApplyResult(_pickStage, _world.Ink, _world.StarsEarned);
499         RankService.Submit(_meta.ClearedCount());
500         ShowVictory(_pickStage, _result);
```

```
501     }
502     else if (_world.RevivesUsed < GameConstants.MaxRevives) ShowRevive();
503     else ShowDefeat(_pickStage, _world.Progress);
504 }
505 void ShowVictory(int stage, ResultInfo info, bool preview = false)
506 {
507     DropOverlay();
508     int next = stage + 1;
509     bool hasNext = next < GameConstants.StageCount && (preview || _meta.Unlocked(next));
510     // 一个广告同时管墨翻倍和当场开这关的宝箱，宝箱位满了或今天开箱广告用完就只翻倍。
511     bool canInk = info.Ink > 0;
512     bool canChest = !preview && info.Chest >= 0 && _meta.HasPendingChest;
513     VictoryPanel panel = null;
514     panel = VictoryPanel.Show(_layer, new VictoryArgs
515     {
516         Stage = stage,
517         Info = info,
518         NextCost = hasNext ? _meta.StageCost(next) : -1,
519         NextAffordable = hasNext && (preview || _meta.CanEnter(next)),
520         Next = preview ? (System.Action)ShowHome : () => StartStage(next),
521         DoubleInk = !canInk && !canChest ? (System.Action)null : () =>
522         {
523             if (_inkDoubled) return;
524             AdStub.Reward("double", () =>
525             {
526                 if (_inkDoubled) return;
527                 _inkDoubled = true;
528                 if (canInk && !preview) _meta.AddInk(info.Ink);
529                 ChestLoot loot = canChest ? _meta.OpenPending() : null;
530                 if (loot == null)
531                 {
532                     if (panel != null) panel.Doubled(false);
533                     return;
534                 }
535                 // 先开箱，收下之后再回结算页滚翻倍的墨，两段演出不抢画面。
536                 ChestOpenView.Show(_layer, _meta, loot, () => { if (panel != null) panel....
537             });
538             },
539             DoubleText = canInk && canChest ? "墨翻倍 + 开宝箱" : canInk ? "墨翻倍" : "立即开宝箱",
540             Home = ShowHome,
541             Forge = () => ShowHome(HomeScreen.TabForge),
542             Skin = skin =>
543             {
544                 ShowHome(HomeScreen.TabForge);
545                 _home?.FocusSkin(skin);
546             }
547         });
548     _overlay = panel.Root;
549 }
550 void ShowRevive()
```

```
551     {
552         DropOverlay();
553         int left = GameConstants.MaxRevives - _world.RevivesUsed;
554         _overlay = RevivePanel.Show(_layer, _world.Progress, left, () =>
555     {
556         AdStub.Reward("revive", () =>
557     {
558         if (_world == null || !_world.TryRevive()) return;
559         DropOverlay();
560         _screen = Screen.Battle;
561     });
562     }, () => ShowDefeat(_pickStage, _world != null ? _world.Progress : 0f)).Root;
563     }
564     void ShowDefeat(int stage, float progress, bool preview = false)
565     {
566         if (!preview)
567             Analytics.LevelFail("hp_zero", Mathf.Clamp(Mathf.RoundToInt(progress * 100f), 0, ...
568         DropOverlay();
569         _overlay = DefeatPanel.Show(_layer, new DefeatArgs
570     {
571         Stage = stage,
572         Progress = progress,
573         RetryCost = _meta.StageCost(stage),
574         CanRetry = preview || _meta.CanEnter(stage),
575         Advice = DefeatAdvice.Build(_meta),
576         Retry = preview ? (System.Action)ShowHome : () => StartStage(stage),
577         Improve = ShowHome,
578         Home = ShowHome
579     }).Root;
580     }
581     // GM 里预览三张结算页，不碰存档。
582     void GmPreview(int kind)
583     {
584         if (_screen != Screen.Lobby) return;
585         int stage = Mathf.Clamp(_meta.ClearedCount(), 0, GameConstants.StageCount - 2);
586         _inkDoubled = false;
587         if (kind == 0)
588         {
589             ShowVictory(stage, new ResultInfo
590         {
591             Ink = 86, DailyDouble = true, Diamonds = 5,
592             Chest = (int)ChestTier.Gold, ChestFull = true, ChestInk = 50, Stars = 3, NewB...
593             NewSkins = new[] { SkinCatalog.Flame, SkinCatalog.Lucky },
594             NewLines = new[] { (int)ForgeLine.BaseHp }
595         }, true);
596         }
597         else if (kind == 1)
598         {
599             DropOverlay();
600             _overlay = RevivePanel.Show(_layer, 0.83f, GameConstants.MaxRevives,
```

```
601         DropOverlay, () => ShowDefeat(stage, 0.83f, true)).Root;
602     }
603     else ShowDefeat(stage, 0.64f, true);
604 }
605 void DropOverlay()
606 {
607     if (_overlay != null) Destroy(_overlay.gameObject);
608     _overlay = null;
609 }
610 void ClearLayer()
611 {
612     DropOverlay();
613     for (int i = _layer.childCount - 1; i >= 0; i--)
614         Destroy(_layer.GetChild(i).gameObject);
615     _hud = null;
616 }
617 void TryCapturePreview()
618 {
619     int page = _world.PreviewPage;
620     if (_world.PreviewAge < 2.2f) return;
621     if (page == _shotPage) return;
622     if (page < 0 || page >= _shotGot.Length) return;
623     _shotPage = page;
624     string dir = Path.GetFullPath(Path.Combine(Application.dataPath, "../game_assets/b...
625     Directory.CreateDirectory(dir);
626     DumpGame(Path.Combine(dir, "page_" + page + ".png"));
627     _shotGot[page] = true;
628     bool all = true;
629     for (int i = 0; i < _shotGot.Length; i++)
630         if (!_shotGot[i]) all = false;
631     if (all)
632         File.WriteAllText(Path.Combine(Application.dataPath, "../Library/ink_shot_preview...
633 }
634 static void DumpGame(string path)
635 {
636     Camera cam = Camera.main;
637     if (cam == null) return;
638     const int w = 720;
639     const int h = 1280;
640     RenderTexture rt = RenderTexture.GetTemporary(w, h, 24);
641     RenderTexture prev = cam.targetTexture;
642     cam.targetTexture = rt;
643     cam.Render();
644     RenderTexture prevActive = RenderTexture.active;
645     RenderTexture.active = rt;
646     var tex = new Texture2D(w, h, TextureFormat.RGB24, false);
647     tex.ReadPixels(new Rect(0, 0, w, h), 0, 0);
648     tex.Apply(false, false);
649     cam.targetTexture = prev;
650     RenderTexture.active = prevActive;
```

```
651     RenderTexture.ReleaseTemporary(rt);
652     File.WriteAllBytes(path, tex.EncodeToPNG());
653     Destroy(tex);
654 }
655 }
656 }
657 // ===== FILE: Assets/Scripts/Core/GameConstants.cs =====
658 namespace InkLine
659 {
660     public static class GameConstants
661     {
662         public const int Columns = 6;
663         public const int Rows = 3;
664         public const int MaxStar = 3;
665         public const int MaxEmitters = 4;
666         // 局里看广告加炮的上限，和列数对齐。局外锻造仍然停在 MaxEmitters。
667         public const int AdEmitterCap = 6;
668         public const int BaseHp = 3;
669         public const int MaxRevives = 2;
670         public const int FirstDraftCost = 4;
671         public const int DraftCostStep = 2;
672         // 八章，每章九关，第九关是章底。
673         // MetaProgress.Normalize 里的 Fit() 会把旧存档的星数组补齐到新长度，
674         // 改这个数字不会打坏已有存档。
675         public const int ChapterSize = 9;
676         public const int Chapters = 8;
677         public const int StageCount = ChapterSize * Chapters;
678         // 局外：体力。每次进关都收，失败不退。见 MetaProgress.StageCost。
679         public const int StaminaMax = 12;
680         public const int StaminaPerStage = 2;
681         public const int StaminaFinale = 3;
682         public const int StaminaRegenMinutes = 15;
683         public const int AdStaminaGain = 6;
684         public const int AdStaminaPerDay = 5;
685         public const int StarterInk = 85;
686         // 新手礼包：看两次激励广告，领一次就没了。附带一个当场打开的银宝箱。
687         public const int GiftAds = 2;
688         public const int GiftInk = 100;
689         public const int GiftDiamond = 30;
690         public const ChestTier GiftChest = ChestTier.Silver;
691         // 七日签到：断一天从第 1 天重来；第一次连签满 7 天送机甲炮（SkinCatalog 2）。
692         // 每天给钻石，第 7 天另给一个金宝箱。
693         public const int CheckDays = 7;
694         public const int CheckStamina = 5;
695         public const int CheckInk = 30;
696         public const int CheckSkin = 2;
697         public static readonly int[] CheckDiamonds = { 3, 3, 4, 4, 5, 5, 20 };
698         // 游戏圈：每天发一条帖子领一次，墨加一个木宝箱。
699         public const int ClubInk = 50;
700         // 首通给钻石。重复通关不给。
```

```

701     public const int DiamondStage = 2;
702     public const int DiamondBoss = 5;
703     public const int DiamondFinale = 15;
704     // 钻石补体力: 10 钻换 10 体力, 每天限次。
705     public const int DiamondStaminaGain = 10;
706     public const int DiamondStaminaPrice = 10;
707     public const int DiamondStaminaPerDay = 3;
708     // 局内墨是这一局拾到的墨, 通关时整笔入账。每只怪掉多少写在 EnemyCatalog 里,
709     // 紫钱袋另给一份。道具不花金币也不花墨。
710     public const int InkMax = 9999;
711     // 道具栏: 开局一格, 通关第三章开第二格, 通关第六章开第三格。值是要通关的章下标。
712     public const int ItemSlots = 3;
713     public static readonly int[] ItemSlotChapter = { -1, 2, 5 };
714     public const float WorldHalfHeight = 8f;
715     public const float CellWidth = 1.16f;
716     public const float CellHeight = 1.16f;
717     // 3 行整块靠下, 上面留给走怪。漏怪线贴在格底下方。
718     public const float GridCenterY = -2.5f;
719     // 炮在格底和底栏之间。太靠下会在 16:9 上被改装键挡住,
720     // 太靠上又会顶到漏怪线。-5.20 给底栏一行按钮留出约一指空隙。
721     public const float EmitterY = -5.20f;
722     public static float LeakY =>
723         GridCenterY - (Rows - 1) * 0.5f * CellHeight - CellHeight * 0.5f - 0.36f;
724     public const float SpawnY = 7.15f;
725     public const float BulletSpeed = 7.2f;
726     // 开局刻意打得慢: 0.34 几乎是机关枪, 两门炮对着一条线就没有瞄准压力。
727     // 往后变快只走局外「射速」线, 不在局里叠。
728     public const float BaseFireInterval = 0.62f;
729     public const float RailSnapSpeed = 14f;
730 }
731 }
732 // ===== FILE: Assets/Scripts/Core/MetaProgress.cs =====
733 using System;
734 using System.Collections.Generic;
735 using UnityEngine;
736 namespace InkLine
737 {
738     public enum ChestState { Empty, Locked, Timing, Ready }
739     [Serializable]
740     public sealed class MetaProgress
741     {
742         const string KeyV2 = "inkline.meta.v2";
743         const string KeyV1 = "inkline.meta.v1";
744         public int Ink;
745         public int Stamina = GameConstants.StaminaMax;
746         public long StaminaTick;
747         public int AdStaminaToday;
748         public int LastDay;
749         public bool DailyWinDone;
750         public bool StarterGranted;

```

```
751     public int[] Stars = new int[GameConstants.StageCount];
752     public int[] Forge = new int[ForgeCatalog.LineCount];
753     public int Skin;
754     public bool[] SkinOwned = new bool[SkinCatalog.Count];
755     // 道具: 等级 0 是还没解锁; 卡是攒着的、还没花掉的张数。
756     public int[] ItemLevel = new int[ItemCatalog.Count];
757     public int[] ItemCards = new int[ItemCatalog.Count];
758     public int[] Equipped = { 0, -1, -1 };
759     public bool ItemStarterDone;
760     // 胜利宝箱位: 0 是空, 其余是 ChestTier + 1。ChestDone 是解锁完成的 UTC 毫秒, 0 表示还没开始解锁。
761     public int[] ChestSlot = new int[ChestCatalog.Slots];
762     public long[] ChestDone = new long[ChestCatalog.Slots];
763     public int ChestCycle;
764     public int PendingChest; // 结算页上还没处理的宝箱, 品阶 + 1; 0 表示没有
765     // 钻石: 只买时间和便利 -- 宝箱立即开、补体力。
766     public int Diamond;
767     public int DiamondStamCount;
768     public int GiftAds;
769     public bool GiftClaimed;
770     public int ClubDay;
771     public int CheckDay;
772     public int CheckRun;
773     public int CheckAdDay;
774     public bool CheckSkinDone;
775     public int CheckLoops;
776     // 图鉴: 位掩码。字按 CardId、词按 WordId、秘卷按 SignaturePairs 下标。
777     // New 系列是「收录了还没点开看过」, 给红点用。
778     public long CodexGlyph;
779     public int CodexWord;
780     public int CodexPair;
781     public long CodexNewGlyph;
782     public int CodexNewWord;
783     public int CodexNewPair;
784     public int CodexEnemy;
785     public int CodexNewEnemy;
786     // 三栏各三枚印, 第 tab*3+i 位表示第 i 枚已经领过。
787     public int CodexMileClaim;
788     // 词条: Seen 是「已经亮过相」, 过关时没亮过的才弹解锁卡;
789     // New 是「亮了还没在炮台页看过」, 给那一行挂「新」。位下标是词条下标。
790     public long ForgeSeen;
791     public long ForgeNew;
792     public int ForgeSeenVer;
793     // 皮肤开放 (到了门槛、可以买) 有没有弹出「新炮台开放」卡。位下标是皮肤下标。
794     public int SkinSeen;
795     // 开放了、炮台页还没去看过的皮肤, 底栏炮台页签挂红点。
796     public int SkinNew;
797     // 活动货币, 留给皮肤专属词条。ForgeCoin.Token 从这里扣。
798     public int Token;
799     public static MetaProgress Load()
800     {
```

```
801     var m = new MetaProgress();
802     string raw = PlayerPrefs.GetString(KeyV2, "");
803     if (!string.IsNullOrEmpty(raw))
804     {
805         try { JsonUtility.FromJsonOverwrite(raw, m); }
806         catch (Exception) { m = new MetaProgress(); }
807     }
808     else m.MigrateV1();
809     m.Normalize();
810     m.GrantStarter();
811     m.Refresh();
812     return m;
813 }
814 // v1 是管道字符串，只有章通关和满星两个布尔。折算成对应的升级等级，
815 // 再按已有星数补一笔墨，免得老存档进来面对空的升级树。
816 void MigrateV1()
817 {
818     string raw = PlayerPrefs.GetString(KeyV1, "");
819     if (string.IsNullOrEmpty(raw)) return;
820     string[] parts = raw.Split('|');
821     if (parts.Length < 2) return;
822     bool chapterCleared = parts[0] == "1";
823     bool goldBonus = parts[1] == "1";
824     if (parts.Length > 2)
825     {
826         string[] stars = parts[2].Split(',');
827         for (int i = 0; i < stars.Length && i < Stars.Length; i++)
828             int.TryParse(stars[i], out Stars[i]);
829     }
830     if (chapterCleared) Forge[(int)ForgeLine.Emitters] = 1;
831     if (goldBonus) Forge[(int)ForgeLine.StartGold] = 1;
832     Ink += ClearedCount() * 12;
833 }
834 void Normalize()
835 {
836     Stars = Fit(Stars, GameConstants.StageCount);
837     Forge = Fit(Forge, ForgeCatalog.LineCount);
838     SkinOwned = Fit(SkinOwned, SkinCatalog.Count);
839     ItemLevel = Fit(ItemLevel, ItemCatalog.Count);
840     ItemCards = Fit(ItemCards, ItemCatalog.Count);
841     if (Equipped == null || Equipped.Length != GameConstants.ItemSlots)
842     {
843         var eq = new int[GameConstants.ItemSlots];
844         for (int s = 0; s < eq.Length; s++) eq[s] = Equipped != null && s < Equipped.Leng...
845         Equipped = eq;
846     }
847     ChestSlot = Fit(ChestSlot, ChestCatalog.Slots);
848     ChestDone = Fit(ChestDone, ChestCatalog.Slots);
849     for (int i = 0; i < Forge.Length; i++)
850         Forge[i] = Mathf.Clamp(Forge[i], 0, ForgeCatalog.MaxLevel(i));
```

```
851     for (int i = 0; i < Stars.Length; i++)
852         Stars[i] = Mathf.Clamp(Stars[i], 0, GameConstants.MaxStar);
853     SkinOwned[0] = true;
854     if (!ItemStarterDone)
855     {
856         // 开局白送一个绿色道具，第一关就能看到「道具自己丢出去」是怎么回事。
857         ItemStarterDone = true;
858         int first = (int)ItemCatalog.Starter;
859         if (ItemLevel[first] <= 0) ItemLevel[first] = 1;
860         for (int s = 0; s < Equipped.Length; s++) Equipped[s] = -1;
861         Equipped[0] = first;
862     }
863     for (int i = 0; i < ItemLevel.Length; i++)
864     {
865         ItemLevel[i] = Mathf.Clamp(ItemLevel[i], 0, ItemCatalog.MaxLevel);
866         ItemCards[i] = Mathf.Max(0, ItemCards[i]);
867     }
868     for (int i = 0; i < ChestSlot.Length; i++)
869     {
870         if (ChestSlot[i] < 0 || ChestSlot[i] > 4) ChestSlot[i] = 0;
871         if (ChestSlot[i] == 0) ChestDone[i] = 0L;
872     }
873     if (PendingChest < 0 || PendingChest > 4) PendingChest = 0;
874     // 结算页上被杀进程，宝箱照常放进宝箱位，不白丢。
875     ResolvePending();
876     Skin = SkinOwned[Mathf.Clamp(Skin, 0, SkinCatalog.Count - 1)] ? Mathf.Clamp(Skin, 0, ...
877     for (int s = 0; s < Equipped.Length; s++)
878     {
879         int id = Equipped[s];
880         if (id < 0 || id >= ItemCatalog.Count || ItemLevel[id] <= 0 || !ItemSlotOpen(s)) ...
881         for (int t = 0; t < s; t++)
882             if (Equipped[t] == Equipped[s]) Equipped[s] = -1;
883     }
884     Ink = Mathf.Max(0, Ink);
885     Diamond = Mathf.Max(0, Diamond);
886     Token = Mathf.Max(0, Token);
887     Stamina = Mathf.Clamp(Stamina, 0, GameConstants.StaminaMax);
888     if (ForgeSeenVer < 1)
889     {
890         // 加解锁卡之前就有的五条，老存档早就见过，不再补弹。
891         for (int i = 0; i < ForgeCatalog.LineCount && i <= (int)ForgeLine.BaseHp; i++)
892             if (ForgeShown(i)) ForgeSeen |= 1L << i;
893         ForgeSeenVer = 1;
894     }
895     if (ForgeSeenVer < 2)
896     {
897         // 暴击、金币收益改成了赤焰、福袋的专属词条，词条本身不再弹卡，跟着皮肤一起亮。
898         // 原有四款皮肤早就摆在炮台上，只给新加的两款补「新炮台开放」。
899         for (int i = 0; i < ForgeCatalog.LineCount; i++)
900             if (ForgeCatalog.Get(i).Exclusive) ForgeSeen |= 1L << i;
```

```
901         for (int i = 0; i < SkinCatalog.Count && i < SkinCatalog.Flame; i++) SkinSeen |= ...
902         ForgeSeenVer = 2;
903     }
904 }
905 // 花墨买的皮肤到了章节门槛，结算页弹「新炮台开放」。买没买另说。
906 public bool SkinOpened(int i)
907 {
908     SkinDef d = SkinCatalog.Get(i);
909     return d.Way == SkinWay.Ink && (d.Chapter < 0 || ChapterCleared(d.Chapter));
910 }
911 // 炮台页上现在就能拿到：看广告，或者章节到了可以花钱。签到和活动不在这一页领。
912 public bool SkinReady(int i)
913 {
914     SkinDef d = SkinCatalog.Get(i);
915     return d.Way == SkinWay.Ad || SkinOpened(i);
916 }
917 int[] TakeSkinReveals()
918 {
919     var got = new List<int>();
920     for (int i = 0; i < SkinCatalog.Count; i++)
921     {
922         int bit = 1 << i;
923         if ((SkinSeen & bit) != 0 || SkinOwned[i] || !SkinOpened(i)) continue;
924         SkinSeen |= bit;
925         SkinNew |= bit;
926         got.Add(i);
927     }
928     return got.ToArray();
929 }
930 static int[] Fit(int[] src, int len)
931 {
932     if (src != null && src.Length == len) return src;
933     var dst = new int[len];
934     if (src != null)
935         for (int i = 0; i < src.Length && i < len; i++) dst[i] = src[i];
936     return dst;
937 }
938 static long[] Fit(long[] src, int len)
939 {
940     if (src != null && src.Length == len) return src;
941     var dst = new long[len];
942     if (src != null)
943         for (int i = 0; i < src.Length && i < len; i++) dst[i] = src[i];
944     return dst;
945 }
946 static bool[] Fit(bool[] src, int len)
947 {
948     if (src != null && src.Length == len) return src;
949     var dst = new bool[len];
950     if (src != null)
```

```
951         for (int i = 0; i < src.Length && i < len; i++) dst[i] = src[i];
952     return dst;
953 }
954 void GrantStarter()
955 {
956     if (StarterGranted) return;
957     StarterGranted = true;
958     Ink += GameConstants.StarterInk;
959     Stamina = GameConstants.StaminaMax;
960     StaminaTick = DateTime.UtcNow.Ticks;
961     Save();
962 }
963 public void Save()
964 {
965     PlayerPrefs.SetString(KeyV2, JsonUtility.ToJson(this));
966     CloudSync.Touch();
967     PlayerPrefs.Save();
968 }
969 // 清档后紧接着 Load 会写出新档并标脏，调用方再 CloudSync.FlushNow，云端一起清。
970 public static void Wipe()
971 {
972     PlayerPrefs.DeleteKey(KeyV2);
973     PlayerPrefs.DeleteKey(KeyV1);
974     PlayerPrefs.Save();
975 }
976 public static bool HasLocal => PlayerPrefs.HasKey(KeyV2) || PlayerPrefs.HasKey(KeyV1);
977 public static string RawLocal => PlayerPrefs.GetString(KeyV2, "");
978 // 云端下行用，不标脏。
979 public static void WriteRaw(string json)
980 {
981     PlayerPrefs.SetString(KeyV2, json);
982     PlayerPrefs.DeleteKey(KeyV1);
983 }
984 public void FillStamina()
985 {
986     Stamina = GameConstants.StaminaMax;
987     StaminaTick = DateTime.UtcNow.Ticks;
988     Save();
989 }
990 // GM: 每个道具都塞 n 张卡。
991 public void GrantCards(int n)
992 {
993     if (n <= 0) return;
994     for (int i = 0; i < ItemCards.Length && i < ItemCatalog.Count; i++)
995         if (ItemLevel[i] < ItemCatalog.MaxLevel) ItemCards[i] += n;
996     Save();
997 }
998 public void UnlockStages()
999 {
1000     for (int i = 0; i < Stars.Length; i++)
```

```
1001     Stars[i] = GameConstants.MaxStar;
1002     Save();
1003 }
1004 public void UnlockSkins()
1005 {
1006     for (int i = 0; i < SkinOwned.Length; i++)
1007         SkinOwned[i] = true;
1008     Save();
1009 }
1010 public void UnlockItems()
1011 {
1012     for (int i = 0; i < ItemLevel.Length && i < ItemCatalog.Count; i++)
1013         if (ItemLevel[i] <= 0) ItemLevel[i] = 1;
1014     Save();
1015 }
1016 public void MaxForge()
1017 {
1018     for (int i = 0; i < Forge.Length && i < ForgeCatalog.LineCount; i++)
1019         Forge[i] = ForgeCatalog.MaxLevel(i);
1020     Save();
1021 }
1022 // 大厅每帧调。体力恢复和日重置都是纯读时间的，放一起省心。
1023 public void Refresh()
1024 {
1025     bool dirty = RollDay();
1026     if (RegenStamina()) dirty = true;
1027     if (TickChests()) dirty = true;
1028     if (dirty) Save();
1029 }
1030 static int Today => DayKey(DateTime.Now);
1031 static int DayKey(DateTime d) => d.Year * 10000 + d.Month * 100 + d.Day;
1032 static int ShiftDay(int key, int days)
1033 {
1034     if (key <= 0) return key;
1035     var d = new DateTime(key / 10000, key / 100 % 100, key % 100);
1036     return DayKey(d.AddDays(days));
1037 }
1038 bool RollDay()
1039 {
1040     int today = Today;
1041     if (LastDay == today) return false;
1042     LastDay = today;
1043     AdStaminaToday = 0;
1044     DiamondStamCount = 0;
1045     DailyWinDone = false;
1046     return true;
1047 }
1048 static long RegenTicks => TimeSpan.TicksPerMinute * GameConstants.StaminaRegenMinutes;
1049 bool RegenStamina()
1050 {
```

```
1051     long now = DateTime.UtcNow.Ticks;
1052     if (StaminaTick <= 0L || Stamina >= GameConstants.StaminaMax)
1053     {
1054         // 满体力时只把锚点贴到当下，不报脏 -- 否则大厅每帧写一次 PlayerPrefs。
1055         // 锚点不落盘也没事：加载时若仍是满的会再贴一次，离开满格由 SpendStamina 落盘。
1056         StaminaTick = now;
1057         return false;
1058     }
1059     // 把系统时间往回调不给恢复，只把锚点拉到当下。往前调挡不住，
1060     // 要彻底堵得靠云存档时间戳，见 docs/局外成长.md。
1061     if (now < StaminaTick)
1062     {
1063         StaminaTick = now;
1064         return true;
1065     }
1066     long gain = (now - StaminaTick) / RegenTicks;
1067     if (gain <= 0L) return false;
1068     int add = (int)Math.Min(gain, GameConstants.StaminaMax);
1069     Stamina = Mathf.Min(GameConstants.StaminaMax, Stamina + add);
1070     StaminaTick = Stamina >= GameConstants.StaminaMax ? now : StaminaTick + add * RegenTi...
1071     return true;
1072 }
1073 public int SecondsToNextStamina()
1074 {
1075     if (Stamina >= GameConstants.StaminaMax) return 0;
1076     long due = StaminaTick + RegenTicks - DateTime.UtcNow.Ticks;
1077     return due <= 0L ? 0 : (int)(due / TimeSpan.TicksPerSecond);
1078 }
1079 // 进关就收，打过没打过一样。失败、中途回首页都不退。
1080 public int StageCost(int stage)
1081 {
1082     if (stage < 0 || stage >= Stars.Length) return 0;
1083     return StageCatalog.Get(stage).StaminaCost;
1084 }
1085 public bool CanEnter(int stage) => Stamina >= StageCost(stage);
1086 public void SpendStamina(int n)
1087 {
1088     if (n <= 0) return;
1089     if (Stamina >= GameConstants.StaminaMax) StaminaTick = DateTime.UtcNow.Ticks;
1090     Stamina = Mathf.Max(0, Stamina - n);
1091     Save();
1092 }
1093 public bool CanAdStamina => AdStaminaToday < GameConstants.AdStaminaPerDay
1094     && Stamina < GameConstants.StaminaMax;
1095 public void GrantAdStamina()
1096 {
1097     AdStaminaToday++;
1098     Stamina = Mathf.Min(GameConstants.StaminaMax, Stamina + GameConstants.AdStaminaGain);
1099     Save();
1100 }
```

```

1101     public bool Unlocked(int stage) => stage <= 0 || Stars[stage - 1] > 0;
1102     // 已通关几关。锻造、皮肤的门槛都按这个数。
1103     public int ClearedCount()
1104     {
1105         int n = 0;
1106         for (int i = 0; i < Stars.Length; i++) if (Stars[i] > 0) n++;
1107         return n;
1108     }
1109     // chapter 从 0 数。
1110     public bool ChapterCleared(int chapter)
1111     {
1112         int last = (chapter + 1) * GameConstants.ChapterSize - 1;
1113         return last >= 0 && last < Stars.Length && Stars[last] > 0;
1114     }
1115     // 技能上的「通关解锁」指通关第三章。皮肤各自写在 SkinDef 上。
1116     public const int ClearGateChapter = 2;
1117     public ForgeStats Forged
1118     {
1119         get
1120         {
1121             return ForgeCatalog.Stats(Forge, Skin);
1122         }
1123     }
1124     // 技能和成词伤害的标尺：一发不带字的炮弹打多少。加成页要按玩家当前的
1125     // 锻造和皮肤把倍数换算成点数显示，所以这里也要有一份。
1126     public float ShotBase =>
1127         (ShotMods.DefaultBase + SkinCatalog.Get(Skin).DamageAdd) * Forged.DamageMul;
1128     public int StartEmitters => Forged.Emitters;
1129     public int StartGold => Forged.StartGold;
1130     public Color SkinTint => SkinCatalog.Get(Skin).Tint;
1131     // 结算的墨就是这一局亲手拾到的墨，关卡不再另发一笔。星级只留历史最高；
1132     // 过没过仍按 Stars > 0 判，旧存档里只有 1 的照样算通关。
1133     public ResultInfo ApplyResult(int stage, int collected, int stars)
1134     {
1135         var r = new ResultInfo { Chest = -1 };
1136         if (stage < 0 || stage >= Stars.Length) return r;
1137         StageDef def = StageCatalog.Get(stage);
1138         stars = Mathf.Clamp(stars, 1, GameConstants.MaxStar);
1139         r.Stars = stars;
1140         r.FirstClear = Stars[stage] <= 0;
1141         r.NewBest = stars > Stars[stage];
1142         if (r.FirstClear)
1143         {
1144             r.Diamonds = def.Finale ? GameConstants.DiamondFinale
1145                 : def.HasBoss ? GameConstants.DiamondBoss : GameConstants.DiamondStage;
1146             Diamond += r.Diamonds;
1147         }
1148         ChestTier tier = ChestCatalog.ForStage(def, r.FirstClear, ChestCycle);
1149         if (!def.Finale && !def.HasBoss) ChestCycle++;
1150         // 宝箱先挂在结算页上：看广告当场开就不占位，离开结算页才放进宝箱位。

```

```
1151     ResolvePending();
1152     PendingChest = (int)tier + 1;
1153     r.Chest = (int)tier;
1154     r.ChestFull = ChestsFull;
1155     r.ChestInk = r.ChestFull ? ChestCatalog.InkAvg(tier) : 0;
1156     if (r.NewBest) Stars[stage] = stars;
1157     r.NewLines = TakeForgeReveals();
1158     r.NewSkins = TakeSkinReveals();
1159     int ink = Mathf.Max(0, collected);
1160     if (!DailyWinDone)
1161     {
1162         DailyWinDone = true;
1163         ink *= 2;
1164         r.DailyDouble = true;
1165     }
1166     Ink += ink;
1167     r.Ink = ink;
1168     Save();
1169     return r;
1170 }
1171 public bool GiftReady => !GiftClaimed && GiftAds >= GameConstants.GiftAds;
1172 public void AddGiftAd()
1173 {
1174     if (GiftClaimed || GiftAds >= GameConstants.GiftAds) return;
1175     GiftAds++;
1176     Save();
1177 }
1178 // 礼包里的银宝箱不进宝箱位，领了当场开。钻石和墨先不入账，等飞进顶栏再 GrantGiftWallet。
1179 // 返回开出来的东西，没领成返回 null。
1180 public ChestLoot ClaimGift()
1181 {
1182     if (!GiftReady) return null;
1183     GiftClaimed = true;
1184     ChestLoot loot = ChestCatalog.Roll(GameConstants.GiftChest, ItemLevel, Equipped);
1185     ApplyLoot(loot);
1186     Save();
1187     return loot;
1188 }
1189 public void GrantGiftWallet()
1190 {
1191     AddInk(GameConstants.GiftInk);
1192     AddDiamond(GameConstants.GiftDiamond);
1193 }
1194 public bool ClubClaimedToday => ClubDay == Today;
1195 // slot 是木宝箱进了哪个位，-1 表示位满了，折成了墨。
1196 public bool ClaimClub(out int slot)
1197 {
1198     slot = -1;
1199     if (ClubClaimedToday) return false;
1200     ClubDay = Today;
```

```
1201     Ink += GameConstants.ClubInk;
1202     slot = GrantChest(ChestTier.Wood);
1203     Save();
1204     return true;
1205 }
1206 public bool CheckedToday => CheckDay == Today;
1207 public bool CheckAdDoneToday => CheckAdDay == Today;
1208 // 这一轮已签几天 (0~7) 。昨天没签、或者上一轮已满 7 天，从 0 算起。
1209 public int CheckShown
1210 {
1211     get
1212     {
1213         if (CheckedToday) return Mathf.Clamp(CheckRun, 0, GameConstants.CheckDays);
1214         bool alive = CheckDay == ShiftDay(Today, -1) && CheckRun < GameConstants.CheckDays;
1215         return alive ? Mathf.Max(0, CheckRun) : 0;
1216     }
1217 }
1218 // 第 7 天是否还送皮肤：没送过，而且玩家还没有那一款。
1219 public bool CheckSkinPending => !CheckSkinDone && !(GameConstants.CheckSkin < SkinOwned.L...
1220         && SkinOwned[GameConstants.CheckSkin]);
1221 public static int CheckDiamondOf(int day)
1222 {
1223     int[] table = GameConstants.CheckDiamonds;
1224     return table[Mathf.Clamp(day, 1, table.Length) - 1];
1225 }
1226 // 返回今天是第几天 (1~7) ，签过了返回 0。skin 表示这一签送了签到皮肤。
1227 // chest 是第 7 天的金宝箱进了哪个位：-2 今天没有宝箱，-1 位满了已折成墨。
1228 public int CheckIn(bool doubled, out bool skin, out int chest)
1229 {
1230     skin = false;
1231     chest = -2;
1232     if (CheckedToday) return 0;
1233     int day = CheckShown + 1;
1234     CheckRun = day;
1235     CheckDay = Today;
1236     int times = doubled ? 2 : 1;
1237     if (doubled) CheckAdDay = Today;
1238     Ink += GameConstants.CheckInk * times;
1239     Diamond += CheckDiamondOf(day) * times;
1240     Stamina = Mathf.Min(GameConstants.StaminaMax, Stamina + GameConstants.CheckStamina * ...
1241     if (day == GameConstants.CheckDays)
1242     {
1243         chest = GrantChest(ChestTier.Gold);
1244         CheckLoops++;
1245         if (!CheckSkinDone)
1246         {
1247             CheckSkinDone = true;
1248             if (GrantSkin(GameConstants.CheckSkin)) skin = true;
1249         }
1250     }
```

```
1251     Save();
1252     return day;
1253 }
1254 // 签完没翻倍的, 当天还能补看一次广告再领一份。
1255 public bool CheckAdBonus()
1256 {
1257     if (!CheckedToday || CheckAdDoneToday) return false;
1258     CheckAdDay = Today;
1259     Ink += GameConstants.CheckInk;
1260     Diamond += CheckDiamondOf(CheckRun);
1261     Stamina = Mathf.Min(GameConstants.StaminaMax, Stamina + GameConstants.CheckStamina);
1262     Save();
1263     return true;
1264 }
1265 // GM: 把签到记录往前挪一天, 等于跨天, 连签不断。
1266 public void GmCheckNextDay()
1267 {
1268     CheckDay = ShiftDay(CheckDay, -1);
1269     CheckAdDay = ShiftDay(CheckAdDay, -1);
1270     Save();
1271 }
1272 // ---- 图鉴 ----
1273 public bool CodexKnows(CodexKind kind, int i)
1274 {
1275     switch (kind)
1276     {
1277         case CodexKind.Glyph: return (CodexGlyph & (1L << i)) != 0;
1278         case CodexKind.Word: return (CodexWord & (1 << i)) != 0;
1279         case CodexKind.Enemy: return (CodexEnemy & (1 << i)) != 0;
1280         default: return (CodexPair & (1 << i)) != 0;
1281     }
1282 }
1283 public bool CodexIsNew(CodexKind kind, int i)
1284 {
1285     switch (kind)
1286     {
1287         case CodexKind.Glyph: return (CodexNewGlyph & (1L << i)) != 0;
1288         case CodexKind.Word: return (CodexNewWord & (1 << i)) != 0;
1289         case CodexKind.Enemy: return (CodexNewEnemy & (1 << i)) != 0;
1290         default: return (CodexNewPair & (1 << i)) != 0;
1291     }
1292 }
1293 public bool CodexHasNew =>
1294     CodexNewGlyph != 0 || CodexNewWord != 0 || CodexNewPair != 0
1295     || CodexNewEnemy != 0 || CodexHasMile;
1296 public bool CodexPairNew => CodexNewPair != 0;
1297 public bool CodexBaseNew => CodexNewGlyph != 0 || CodexNewWord != 0;
1298 public bool CodexEnemyNew => CodexNewEnemy != 0;
1299 public bool CodexTabNew(CodexTab tab)
1300 {
```

```

1301     switch (tab)
1302     {
1303         case CodexTab.Pair: return CodexPairNew || TabHasMile(tab);
1304         case CodexTab.Enemy: return CodexEnemyNew || TabHasMile(tab);
1305         default: return CodexBaseNew || TabHasMile(tab);
1306     }
1307 }
1308 public bool CodexHasMile =>
1309     TabHasMile(CodexTab.Glyph) || TabHasMile(CodexTab.Pair) || TabHasMile(CodexTab.Enemy);
1310 public bool TabHasMile(CodexTab tab)
1311 {
1312     IReadOnlyList<CodexEntry> list = CodexCatalog.Of(tab);
1313     int known = CodexCatalog.CountKnown(this, list);
1314     CodexMile[] miles = CodexCatalog.Miles(tab);
1315     for (int i = 0; i < miles.Length; i++)
1316         if (known >= miles[i].Need && !MileClaimed(tab, i)) return true;
1317     return false;
1318 }
1319 public bool MileClaimed(CodexTab tab, int i) =>
1320     (CodexMileClaim & (1 << ((int)tab * 3 + i))) != 0;
1321 // 领到了返回墨数，还没到或已经领过返回 0。
1322 public int ClaimMile(CodexTab tab, int i)
1323 {
1324     CodexMile[] miles = CodexCatalog.Miles(tab);
1325     if (i < 0 || i >= miles.Length) return 0;
1326     if (MileClaimed(tab, i)) return 0;
1327     if (CodexCatalog.CountKnown(this, CodexCatalog.Of(tab)) < miles[i].Need) return 0;
1328     CodexMileClaim |= 1 << ((int)tab * 3 + i);
1329     AddInk(miles[i].Ink);
1330     return miles[i].Ink;
1331 }
1332 // 第一次收录返回 true。
1333 public bool CodexLearn(CodexKind kind, int i)
1334 {
1335     if (CodexKnows(kind, i)) return false;
1336     switch (kind)
1337     {
1338         case CodexKind.Glyph: CodexGlyph |= 1L << i; CodexNewGlyph |= 1L << i; break;
1339         case CodexKind.Word: CodexWord |= 1 << i; CodexNewWord |= 1 << i; break;
1340         case CodexKind.Enemy: CodexEnemy |= 1 << i; CodexNewEnemy |= 1 << i; break;
1341         default: CodexPair |= 1 << i; CodexNewPair |= 1 << i; break;
1342     }
1343     Save();
1344     return true;
1345 }
1346 public void CodexSeen(CodexKind kind, int i)
1347 {
1348     if (!CodexIsNew(kind, i)) return;
1349     switch (kind)
1350     {

```

```
1351         case CodexKind.Glyph: CodexNewGlyph &= ~(1L << i); break;
1352         case CodexKind.Word: CodexNewWord &= ~(1 << i); break;
1353         case CodexKind.Enemy: CodexNewEnemy &= ~(1 << i); break;
1354         default: CodexNewPair &= ~(1 << i); break;
1355     }
1356     Save();
1357 }
1358 public void UnlockCodex()
1359 {
1360     for (int i = 1; i < CardCatalog.IdCount; i++)
1361         if (CardCatalog.Get((CardId)i).Wake != CardWake.WordPart) CodexLearn(CodexKind.Gl...
1362     for (int i = (int)WordId.InstantKill; i <= (int)WordId.Cleave; i++) CodexLearn(CodexK...
1363     for (int i = 0; i < SignaturePairs.All.Length; i++) CodexLearn(CodexKind.Pair, i);
1364     for (int i = 0; i < EnemyCatalog.IdCount; i++) CodexLearn(CodexKind.Enemy, i);
1365 }
1366 public void AddInk(int n)
1367 {
1368     if (n <= 0) return;
1369     Ink += n;
1370     Save();
1371 }
1372 public int ForgeLevel(int line) => Forge[Mathf.Clamp(line, 0, ForgeCatalog.LineCount - 1)];
1373 // 炮台页上有没有这一行。通用看通关数，专属看有没有那款皮肤。
1374 public bool ForgeShown(int line)
1375 {
1376     ForgeDef d = ForgeCatalog.Get(line);
1377     int lv = ForgeLevel(line);
1378     if (!d.Exclusive) return ForgeCatalog.Exposed(line, ClearedCount(), lv);
1379     return lv > 0 || (d.Skin >= 0 && d.Skin < SkinOwned.Length && SkinOwned[d.Skin]);
1380 }
1381 public bool ForgeFresh(int line) => (ForgeNew & (1L << line)) != 0;
1382 // 炮台页看过就摘掉「新」。返回摘之前的那份，好让这一屏还能接着显示。
1383 public long TakeForgeNew()
1384 {
1385     long fresh = ForgeNew;
1386     if (fresh == 0) return 0;
1387     ForgeNew = 0;
1388     Save();
1389     return fresh;
1390 }
1391 // 刚亮相、还没弹过解锁卡的词条。按炮台页的排列给，取了就算弹过。
1392 int[] TakeForgeReveals()
1393 {
1394     var got = new List<int>();
1395     for (int i = 0; i < ForgeCatalog.LineCount; i++)
1396     {
1397         long bit = 1L << i;
1398         if ((ForgeSeen & bit) != 0 || !ForgeShown(i)) continue;
1399         ForgeSeen |= bit;
1400         ForgeNew |= bit;
```

```

1401     got.Add(i);
1402 }
1403 got.Sort((a, b) => ForgeCatalog.Rank(a).CompareTo(ForgeCatalog.Rank(b)));
1404 return got.ToArray();
1405 }
1406 public int Wallet(ForgeCoin coin) => coin == ForgeCoin.Token ? Token : Ink;
1407 static string CoinName(ForgeCoin coin) => coin == ForgeCoin.Token ? "活动币" : "墨";
1408 // 买不起 / 没到门槛时 why 里放要显示给玩家的那句话。
1409 public bool CanBuyForge(int line, out string why)
1410 {
1411     int lv = ForgeLevel(line);
1412     ForgeDef d = ForgeCatalog.Get(line);
1413     if (!ForgeShown(line)) { why = "未开放"; return false; }
1414     if (lv >= d.MaxLevel) { why = "已满级"; return false; }
1415     int gate = ForgeCatalog.Gate(line, lv);
1416     if (gate > GameConstants.StageCount)
1417     {
1418         why = string.IsNullOrEmpty(d.LockNote) ? "暂未开放" : d.LockNote;
1419         return false;
1420     }
1421     if (ClearedCount() < gate) { why = $"通关 {gate} 关"; return false; }
1422     int cost = ForgeCatalog.Cost(line, lv);
1423     int have = Wallet(d.Coin);
1424     if (have < cost) { why = $"差 {cost - have} {CoinName(d.Coin)}"; return false; }
1425     why = "";
1426     return true;
1427 }
1428 public bool BuyForge(int line)
1429 {
1430     if (!CanBuyForge(line, out _)) return false;
1431     int cost = ForgeCatalog.Cost(line, ForgeLevel(line));
1432     if (ForgeCatalog.Get(line).Coin == ForgeCoin.Token) Token -= cost;
1433     else Ink -= cost;
1434     Forge[line]++;
1435     Save();
1436     return true;
1437 }
1438 public bool CanBuySkin(int i, out string why)
1439 {
1440     SkinDef d = SkinCatalog.Get(i);
1441     if (i < 0 || i >= SkinOwned.Length || SkinOwned[i]) { why = ""; return false; }
1442     if (d.Way != SkinWay.Ink) { why = SkinCatalog.LockText(d); return false; }
1443     if (d.Chapter >= 0 && !ChapterCleared(d.Chapter)) { why = SkinCatalog.LockText(d); re...
1444     if (Ink < d.Price) { why = $"差 {d.Price - Ink} 墨"; return false; }
1445     why = "";
1446     return true;
1447 }
1448 public bool BuySkin(int i)
1449 {
1450     if (!CanBuySkin(i, out _)) return false;

```

```
1451     Ink -= SkinCatalog.Get(i).Price;
1452     return GrantSkin(i);
1453 }
1454 // 广告、签到、活动发皮肤都走这里。到手就换上，专属词条跟着挂「新」。
1455 public bool GrantSkin(int i)
1456 {
1457     if (i < 0 || i >= SkinCatalog.Count) return false;
1458     SkinOwned = Fit(SkinOwned, SkinCatalog.Count);
1459     if (SkinOwned[i]) return false;
1460     SkinOwned[i] = true;
1461     Skin = i;
1462     SkinSeen |= 1 << i;
1463     SkinNew |= 1 << i;
1464     for (int k = 0; k < ForgeCatalog.LineCount; k++)
1465     {
1466         ForgeDef d = ForgeCatalog.Get(k);
1467         if (!d.Exclusive || d.Skin != i) continue;
1468         ForgeSeen |= 1L << k;
1469         ForgeNew |= 1L << k;
1470     }
1471     Save();
1472     return true;
1473 }
1474 public void EquipSkin(int i)
1475 {
1476     if (i < 0 || i >= SkinCatalog.Count || !SkinOwned[i]) return;
1477     Skin = i;
1478     Save();
1479 }
1480 // ---- 道具 ----
1481 public int ItemRank(int i) =>
1482     ItemLevel != null && i >= 0 && i < ItemLevel.Length ? ItemLevel[i] : 0;
1483 public int ItemCardCount(int i) =>
1484     ItemCards != null && i >= 0 && i < ItemCards.Length ? ItemCards[i] : 0;
1485 // 解锁只看卡，升级要卡和墨。缺什么 why 里写给玩家看。
1486 public bool CanUpgradeItem(int i, out string why)
1487 {
1488     if (i < 0 || i >= ItemCatalog.Count) { why = ""; return false; }
1489     int rank = ItemRank(i);
1490     if (rank >= ItemCatalog.MaxLevel) { why = "已满级"; return false; }
1491     ItemDef d = ItemCatalog.Get(i);
1492     int need = ItemCatalog.NextCards(d, rank);
1493     int have = ItemCardCount(i);
1494     if (have < need) { why = have + "/" + need; return false; }
1495     int price = ItemCatalog.NextPrice(d, rank);
1496     if (Ink < price) { why = $"差 {price - Ink} 墨"; return false; }
1497     why = "";
1498     return true;
1499 }
1500 public bool UpgradeItem(int i)
```

```
24503     }
24504     void OnApplicationPause(bool paused)
24505     {
24506         if (paused && !_hideHooked) FlushNow("pause");
24507     }
24508     void LoadMeta()
24509     {
24510         string raw = PlayerPrefs.GetString(CloudConfig.SyncMetaKey, "");
24511         _m = null;
24512         if (!string.IsNullOrEmpty(raw))
24513         {
24514             try { _m = JsonUtility.FromJson<SyncMeta>(raw); }
24515             catch (Exception) { _m = null; }
24516         }
24517         if (_m == null) _m = new SyncMeta();
24518     }
24519     void WriteMeta() => PlayerPrefs.SetString(CloudConfig.SyncMetaKey, JsonUtility.ToJson(_m));
24520     static string Fingerprint()
24521     {
24522         string s = SystemInfo.deviceModel + "|" + SystemInfo.operatingSystem;
24523         return s.Length > 160 ? s.Substring(0, 160) : s;
24524     }
24525 }
24526 }
24527 // ===== FILE: Assets/Scripts/Platform/GameClubService.cs =====
24528 using System;
24529 using System.Collections;
24530 using UnityEngine;
24531 namespace InkLine
24532 {
24533     // 游戏圈每日发帖，照 hotspot: wx.getGameClubData 拿加密的「今日发帖数」，
24534     // 云函数用登录时存下的 session_key 解开。session_key 过期或还没存（老 token）就重登一次再拉。
24535     // 编辑器里没有游戏圈，当作已发帖，方便验领奖。
24536     public sealed class GameClubService : MonoBehaviour
24537     {
24538         const int DailyPostType = 6;
24539         [Serializable]
24540         sealed class DailyBody
24541         {
24542             public string encryptedData;
24543             public string iv;
24544         }
24545         static GameClubService _it;
24546         static bool _hooked;
24547         public static event Action Shown;
24548         static GameClubService It
24549         {
24550             get
24551             {
24552                 if (_it != null) return _it;
```

```
24553     var go = new GameObject("GameClubService");
24554     DontDestroyOnLoad(go);
24555     _it = go.AddComponent<GameClubService>();
24556     return _it;
24557 }
24558 }
24559 // 从游戏圈切回游戏时通知。只注册一次，面板自己订阅 / 退订 Shown。
24560 public static void Hook()
24561 {
24562     if (_hooked) return;
24563     _hooked = true;
24564     WxBridge.OnShow() => Shown?.Invoke();
24565 }
24566 // done(今日发帖数, err): err 为 null 表示查到了。
24567 public static void DailyPosts(Action<int, string> done)
24568 {
24569     if (!WxBridge.CanUseClub)
24570     {
24571         if (Application.isEditor) done(1, null);
24572         else done(0, "not wx");
24573         return;
24574     }
24575     It.StartCoroutine(It.Fetch(done));
24576 }
24577 IEnumerator Fetch(Action<int, string> done)
24578 {
24579     if (WxBridge.CanUseClub && !WxBridge.PrivacyAgreed)
24580     {
24581         bool settled = false, agreed = false;
24582         WxBridge.EnsurePrivacy(ok => { agreed = ok; settled = true; });
24583         float privacyUntil = Time.realtimeSinceStartup + 20f;
24584         while (!settled && Time.realtimeSinceStartup < privacyUntil) yield return null;
24585         if (!agreed)
24586         {
24587             done(0, "需同意隐私协议后才能同步游戏圈发帖");
24588             yield break;
24589         }
24590     }
24591     for (int attempt = 0; attempt < 2; attempt++)
24592     {
24593         string enc = null, iv = null, err = null;
24594         bool back = false;
24595         WxBridge.GameClubData(DailyPostType, (e, i, x) => { enc = e; iv = i; err = x; bac...
24596         float until = Time.realtimeSinceStartup + CloudConfig.RequestTimeoutSec;
24597         while (!back && Time.realtimeSinceStartup < until) yield return null;
24598         if (!back) { done(0, "getGameClubData timeout"); yield break; }
24599         if (err != null) { done(0, err); yield break; }
24600         CloudReply reply = null;
24601         string callErr = null;
24602         string json = JsonUtility.ToJson(new DailyBody { encryptedData = enc, iv = iv });
```

```
24603     yield return Backend.Call(CloudConfig.GameClubDailyPath, json, (s, r, e) => { rep...
24604     if (callErr == null)
24605     {
24606         done(reply.data != null ? reply.data.postCount : 0, null);
24607         yield break;
24608     }
24609     string code = reply != null ? reply.code : null;
24610     bool stale = code == "NO_WX_SESSION" || code == "DECRYPT_FAIL";
24611     if (!stale || attempt > 0) { done(0, callErr); yield break; }
24612     // 先重登换新 session_key, 再重新拉加密数据; 顺序反了新数据还是旧 key 加的密。
24613     Backend.ClearToken();
24614     string loginErr = null;
24615     yield return Backend.EnsureToken(e => loginErr = e);
24616     if (loginErr != null) { done(0, loginErr); yield break; }
24617 }
24618 }
24619 }
24620 }
24621 // ===== FILE: Assets/Scripts/Platform/RankService.cs =====
24622 using System;
24623 using System.Collections;
24624 using System.Collections.Generic;
24625 using UnityEngine;
24626 using UnityEngine.Networking;
24627 namespace InkLine
24628 {
24629     // 通关排行榜, 逻辑照 hotpot 的 RankService:
24630     // 每次通关报一次已通关关数, 同样的「关数 + 昵称头像」只报一次;
24631     // 打开榜单先把没报上去的补报, 再拉前 N 名和自己的名次。
24632     // 服务端只升不降, 所以本地清档、换设备都不会把榜上成绩拉下来。
24633     public sealed class RankService : MonoBehaviour
24634     {
24635         [Serializable]
24636         sealed class SubmitBody
24637         {
24638             public int cleared;
24639             public string displayName;
24640             public string avatarUrl;
24641         }
24642         [Serializable]
24643         sealed class ListBody
24644         {
24645             public int limit;
24646         }
24647         [Serializable]
24648         sealed class Profile
24649         {
24650             public string nick;
24651             public string avatar;
24652         }
```

```

24653     public sealed class Board
24654     {
24655         public RankRow[] List;
24656         public RankRow Mine;
24657     }
24658     static RankService _it;
24659     static Profile _profile;
24660     readonly Dictionary<string, Sprite> _avatars = new Dictionary<string, Sprite>();
24661     readonly Dictionary<string, Action<Sprite>> _avatarWait = new Dictionary<string, Action<S...
24662     bool _sending;
24663     static RankService It
24664     {
24665         get
24666         {
24667             if (_it != null) return _it;
24668             var go = new GameObject("RankService");
24669             DontDestroyOnLoad(go);
24670             _it = go.AddComponent<RankService>();
24671             return _it;
24672         }
24673     }
24674     // 开发者工具里有 GM 一键全通关，报上去就挂榜首了。编辑器走 anon 身份，服务端不给真机玩家看。
24675     static bool Muted => WxBridge.IsSimulator && !Application.isEditor;
24676     public static bool HasProfile => !string.IsNullOrEmpty(LoadProfile().nick);
24677     public static string Nick => LoadProfile().nick ?? "";
24678     public static string Avatar => LoadProfile().avatar ?? "";
24679     static Profile LoadProfile()
24680     {
24681         if (_profile != null) return _profile;
24682         string raw = PlayerPrefs.GetString(CloudConfig.ProfileKey, "");
24683         try { _profile = string.IsNullOrEmpty(raw) ? null : JsonUtility.FromJson<Profile>(raw...
24684         catch (Exception) { _profile = null; }
24685         if (_profile == null) _profile = new Profile();
24686         return _profile;
24687     }
24688     // 微信拒绝授权也会回一个「微信用户」+ 灰头像，当没给。
24689     public static bool SetProfile(string nick, string avatar)
24690     {
24691         nick = (nick ?? "").Trim();
24692         if (nick.Length == 0 || nick == "微信用户") return false;
24693         _profile = new Profile { nick = nick, avatar = avatar ?? "" };
24694         PlayerPrefs.SetString(CloudConfig.ProfileKey, JsonUtility.ToJson(_profile));
24695         PlayerPrefs.Save();
24696         return true;
24697     }
24698     static string SentKey(int cleared) => cleared + ":" + Nick + ":" + Avatar;
24699     public static void Submit(int cleared)
24700     {
24701         if (cleared <= 0 || Muted) return;
24702         if (PlayerPrefs.GetString(CloudConfig.RankSentKey, "") == SentKey(cleared)) return;

```

```

24703     It.StartCoroutine(It.SubmitRoutine(cleared, null));
24704 }
24705 public static void Load(int cleared, Action<Board, string> done)
24706 {
24707     It.StartCoroutine(It.LoadRoutine(cleared, done));
24708 }
24709 IEnumerator SubmitRoutine(int cleared, Action done)
24710 {
24711     while (_sending) yield return null;
24712     string key = SentKey(cleared);
24713     if (cleared <= 0 || Muted || PlayerPrefs.GetString(CloudConfig.RankSentKey, "") == key)
24714     {
24715         done?.Invoke();
24716         yield break;
24717     }
24718     _sending = true;
24719     var body = new SubmitBody { cleared = cleared, displayName = Nick, avatarUrl = Avatar };
24720     string err = null;
24721     yield return Backend.Call(CloudConfig.RankSubmitPath, JsonUtility.ToJson(body), (s, r...
24722     _sending = false;
24723     if (err == null)
24724     {
24725         PlayerPrefs.SetString(CloudConfig.RankSentKey, key);
24726         PlayerPrefs.Save();
24727     }
24728     else Debug.LogWarning("[Rank] submit failed: " + err);
24729     done?.Invoke();
24730 }
24731 IEnumerator LoadRoutine(int cleared, Action<Board, string> done)
24732 {
24733     yield return SubmitRoutine(cleared, null);
24734     CloudReply reply = null;
24735     string err = null;
24736     var body = new ListBody { limit = CloudConfig.RankListLimit };
24737     yield return Backend.Call(CloudConfig.RankListPath, JsonUtility.ToJson(body), (s, r, ...
24738     if (err != null || reply == null || reply.data == null)
24739     {
24740         done(null, err ?? "empty reply");
24741         yield break;
24742     }
24743     // JsonUtility 不会把缺省的对象字段留成 null, 没上榜时 mine 是个 rank 为 0 的空壳。
24744     RankRow mine = reply.data.mine != null && reply.data.mine.rank > 0 ? reply.data.mine ...
24745     done(new Board { List = reply.data.list ?? new RankRow[0], Mine = mine }, null);
24746 }
24747 // 头像按 URL 缓存。微信头像域名要加进小程序后台的 request / downloadFile 合法域名。
24748 public static void AvatarOf(string url, Action<Sprite> done)
24749 {
24750     if (string.IsNullOrEmpty(url)) { done(null); return; }
24751     var s = It;
24752     if (s._avatars.TryGetValue(url, out Sprite hit)) { done(hit); return; }

```

```

24753         if (s._avatarWait.ContainsKey(url)) { s._avatarWait[url] += done; return; }
24754         s._avatarWait[url] = done;
24755         s.StartCoroutine(s.FetchAvatar(url));
24756     }
24757     IEnumerator FetchAvatar(string url)
24758     {
24759         Sprite sprite = null;
24760         using (var req = UnityWebRequestTexture.GetTexture(url, true))
24761         {
24762             req.timeout = CloudConfig.RequestTimeoutSec;
24763             yield return req.SendWebRequest();
24764             if (req.result == UnityWebRequest.Result.Success)
24765             {
24766                 Texture2D tex = DownloadHandlerTexture.GetContent(req);
24767                 if (tex != null)
24768                 {
24769                     tex.wrapMode = TextureWrapMode.Clamp;
24770                     sprite = Sprite.Create(tex, new Rect(0, 0, tex.width, tex.height), new Ve...
24771                 }
24772             }
24773         }
24774         if (sprite != null) _avatars[url] = sprite;
24775         if (_avatarWait.TryGetValue(url, out Action<Sprite> wait))
24776         {
24777             _avatarWait.Remove(url);
24778             wait?.Invoke(sprite);
24779         }
24780     }
24781 }
24782 }
24783 // ===== FILE: Assets/Scripts/Platform/WxBridge.cs =====
24784 using System;
24785 using UnityEngine;
24786 namespace InkLine
24787 {
24788     public static class WxBridge
24789     {
24790         static Type WxType()
24791         {
24792             return Type.GetType("WeChatWASM.WX, Wx") ?? Type.GetType("WeChatWASM.WX");
24793         }
24794         public static void InitSdk(Action ready)
24795         {
24796             var wx = WxType();
24797             var init = wx != null ? wx.GetMethod("InitSDK", new[] { typeof(Action<int>) }) : null;
24798             if (init == null)
24799             {
24800                 ready();
24801                 return;
24802             }

```

```
24803     Action<int> cb = _ => ready();
24804     init.Invoke(null, new object[] { cb });
24805 }
24806 // wx.login 拿一次性 code, 换 openid 在云函数里做。非微信构建直接报失败。
24807 public static void Login(Action<string> ok, Action<string> fail)
24808 {
24809 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24810     WeChatWASM.WX.Login(new WeChatWASM.LoginOption
24811     {
24812         success = r => ok(r.code),
24813         fail = e => fail(e.errMsg)
24814     });
24815 #else
24816     fail("not minigame");
24817 #endif
24818 }
24819 // 切后台（锁屏、回桌面、聊天顶部）。编辑器和其它平台由调用方走 OnApplicationPause。
24820 public static bool OnHide(Action hide)
24821 {
24822 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24823     if (Application.isEditor) return false;
24824     WeChatWASM.WX.OnHide(_ => hide());
24825     return true;
24826 #else
24827     return false;
24828 #endif
24829 }
24830 // 真机上才有微信原生的授权按钮；编辑器和开发者工具以外的平台一律没有。
24831 public static bool CanAskProfile
24832 {
24833     get
24834     {
24835 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24836         return !Application.isEditor;
24837 #else
24838         return false;
24839 #endif
24840     }
24841 }
24842 // 后台改过隐私协议之后，昵称和游戏圈数据都要先过这一关，才会弹出微信自己的隐私弹窗。
24843 // 同意过的这次会话里不再问。编辑器没有这套接口，直接当已同意。
24844 public static bool PrivacyAgreed { get; private set; }
24845 static bool _privacyBusy;
24846 static Action<bool> _privacyWait;
24847 public static void EnsurePrivacy(Action<bool> done)
24848 {
24849     if (!CanAskProfile || PrivacyAgreed)
24850     {
24851         if (!CanAskProfile) PrivacyAgreed = true;
24852         done?.Invoke(true);
24853     }
24854 }
```

```

24853         return;
24854     }
24855     _privacyWait += done;
24856     if (_privacyBusy) return;
24857     _privacyBusy = true;
24858     #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24859     try
24860     {
24861         WeChatWASM.WX.RequirePrivacyAuthorize(new WeChatWASM.RequirePrivacyAuthorizeOption
24862         {
24863             success = _ => FinishPrivacy(true),
24864             fail = _ => FinishPrivacy(false),
24865         });
24866     }
24867     catch (Exception e)
24868     {
24869         Debug.LogWarning("[Privacy] " + e.Message);
24870         FinishPrivacy(true);
24871     }
24872     #else
24873     FinishPrivacy(true);
24874     #endif
24875 }
24876 static void FinishPrivacy(bool ok)
24877 {
24878     _privacyBusy = false;
24879     if (ok) PrivacyAgreed = true;
24880     Action<bool> wait = _privacyWait;
24881     _privacyWait = null;
24882     wait?.Invoke(ok);
24883 }
24884 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24885 static WeChatWASM.WX.UserInfoButton _infoBtn;
24886 #endif
24887 // 微信只允许用它自己画的透明按钮拿昵称头像，所以在 Unity 按钮正上方盖一个同样大小的。
24888 // SDK 里还会把坐标除一次 devicePixelRatio，所以这里传物理像素，不能传逻辑像素。
24889 // got(nick, avatarUrl, err): 没拿到时 nick 为空，err 是微信给的 errMsg。
24890 public static void ShowProfileButton(Rect screenRect, Action<string, string, string> got)
24891 {
24892     HideProfileButton();
24893     #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24894     if (Application.isEditor) return;
24895     RectInt r = ToPhysical(screenRect);
24896     _infoBtn = WeChatWASM.WX.CreateUserInfoButton(r.x, r.y, r.width, r.height, "zh_CN", f...
24897     _infoBtn.OnTap(res =>
24898     {
24899         // userInfo 是结构体，不能和 null 写在同一个三元表达式里。
24900         if (res == null || string.IsNullOrEmpty(res.userInfo.nickName))
24901         {
24902             string err = res == null ? "no response" : (res.errCode + " " + res.errMsg);

```

```

24903         Debug.LogWarning("[Profile] " + err);
24904         got(null, null, err);
24905         return;
24906     }
24907     got(res.userInfo.nickName, res.userInfo.avatarUrl ?? "", null);
24908 });
24909 _infoBtn.Show();
24910 #endif
24911 }
24912 // 拒绝过一次之后，微信不会再弹授权框，原生按钮点了直接失败。只能引去设置页重新打开。
24913 // 弹窗的「去设置」也算用户点击，openSetting 才放行。done(true) 表示回来时已允许。
24914 public static void AskOpenSetting(string content, Action<bool> done)
24915 {
24916     #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24917         if (!Application.isEditor)
24918         {
24919             WeChatWASM.WX.ShowModal(new WeChatWASM.ShowModalOption
24920             {
24921                 title = "需要你的授权",
24922                 content = content,
24923                 confirmText = "去设置",
24924                 cancelText = "取消",
24925                 showCancel = true,
24926                 success = m =>
24927                 {
24928                     if (!m.confirm) { done?.Invoke(false); return; }
24929                     WeChatWASM.WX.OpenSetting(new WeChatWASM.OpenSettingOption
24930                     {
24931                         success = s =>
24932                         {
24933                             bool ok = s.authSetting != null && s.authSetting.ContainsKey("sco...
24934                                 && s.authSetting["scope.userInfo"];
24935                             done?.Invoke(ok);
24936                         },
24937                         fail = _ => done?.Invoke(false),
24938                     });
24939                 },
24940                 fail = _ => done?.Invoke(false),
24941             });
24942             return;
24943         }
24944     #endif
24945     done?.Invoke(false);
24946 }
24947 public static void HideProfileButton()
24948 {
24949     #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24950         if (_infoBtn == null) return;
24951         _infoBtn.Destroy();
24952         _infoBtn = null;

```

```
24953 #endif
24954     }
24955     // 游戏圈入口也只能用微信原生按钮打开，同样透明地盖在 Unity 按钮上。
24956     public static bool CanUseClub => CanAskProfile;
24957 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24958     static WeChatWASM.WXGameClubButton _clubBtn;
24959 #endif
24960     public static void ShowClubButton(Rect screenRect)
24961     {
24962         HideClubButton();
24963 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24964         if (Application.isEditor) return;
24965         RectInt r = ToWindow(screenRect);
24966         _clubBtn = WeChatWASM.WX.CreateGameClubButton(new WeChatWASM.WXCreateGameClubButtonParam
24967         {
24968             type = WeChatWASM.GameClubButtonType.text,
24969             text = " ",
24970             style = new WeChatWASM.GameClubButtonStyle
24971             {
24972                 left = r.x,
24973                 top = r.y,
24974                 width = r.width,
24975                 height = r.height,
24976                 backgroundColor = "rgba(0,0,0,0)",
24977                 borderColor = "rgba(0,0,0,0)",
24978                 borderWidth = 0,
24979                 borderRadius = 0,
24980                 color = "rgba(0,0,0,0)",
24981                 fontSize = 12,
24982                 lineHeight = r.height,
24983             },
24984         });
24985         _clubBtn?.Show();
24986 #endif
24987     }
24988     public static void HideClubButton()
24989     {
24990 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
24991         if (_clubBtn == null) return;
24992         _clubBtn.Destroy();
24993         _clubBtn = null;
24994 #endif
24995     }
24996     // 游戏圈数据是加密的，用登录时的 session_key 在云函数里解。
24997     // done(encryptedData, iv, err): err 为 null 表示拿到了。
24998     public static void GameClubData(int dataType, Action<string, string, string> done)
24999     {
25000 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
25001         if (!Application.isEditor)
25002         {
```

```

25003         WeChatWASM.WX.GetGameClubData(new WeChatWASM.GetGameClubDataOption
25004         {
25005             dataTypeList = new[] { new WeChatWASM.DataType { type = dataType } },
25006             success = r => done(r.encryptedData, r.iv, string.IsNullOrEmpty(r.encryptedDa...
25007             fail = e => done(null, null, e != null ? e.errMsg : "fail"),
25008         });
25009         return;
25010     }
25011 #endif
25012     done(null, null, "not minigame");
25013 }
25014 // 从游戏圈切回来时刷新进度。编辑器里没有，返回 false。
25015 public static bool OnShow(Action show)
25016 {
25017 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
25018     if (Application.isEditor) return false;
25019     WeChatWASM.WX.OnShow(_ => show());
25020     return true;
25021 #else
25022     return false;
25023 #endif
25024 }
25025 // 游戏圈按钮的 style 是逻辑像素，SDK 不再缩放。
25026 static RectInt ToWindow(Rect screenRect)
25027 {
25028     float winW = Num(SystemInfo(), "windowWidth");
25029     float k = winW > 0f ? winW / Mathf.Max(1, Screen.width) : 1f;
25030     return TopLeft(screenRect, k);
25031 }
25032 // 昵称按钮的 JS 会再除 devicePixelRatio。Unity 屏幕若已是逻辑像素，这里先乘回去。
25033 static RectInt ToPhysical(Rect screenRect)
25034 {
25035     float winW = Num(SystemInfo(), "windowWidth");
25036     float dpr = Num(SystemInfo(), "pixelRatio");
25037     if (dpr < 1f) dpr = 1f;
25038     float scale = winW > 0f && Screen.width < winW * 1.5f ? dpr : 1f;
25039     return TopLeft(screenRect, scale);
25040 }
25041 static RectInt TopLeft(Rect screenRect, float scale)
25042 {
25043     return new RectInt(
25044         Mathf.RoundToInt(screenRect.xMin * scale),
25045         Mathf.RoundToInt((Screen.height - screenRect.yMax) * scale),
25046         Mathf.Max(1, Mathf.RoundToInt(screenRect.width * scale)),
25047         Mathf.Max(1, Mathf.RoundToInt(screenRect.height * scale)));
25048 }
25049 // 玩家昵称什么字都可能，游戏字体只切了用到的几百个字。
25050 // 真机拿微信的系统字体；编辑器里用本机字体。拿不到回调 null。
25051 static Font _sysFont;
25052 static bool _sysFontAsked;

```

```

25053     static Action<Font> _sysFontWait;
25054     public static void SystemFont(Action<Font> done)
25055     {
25056         if (_sysFont != null || (_sysFontAsked && _sysFontWait == null)) { done(_sysFont); re...
25057         _sysFontWait += done;
25058         if (_sysFontAsked) return;
25059         _sysFontAsked = true;
25060 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
25061         if (!Application.isEditor)
25062         {
25063             WeChatWASM.WX.GetWXFont(null, f => FinishFont(f));
25064             return;
25065         }
25066 #endif
25067         FinishFont(Font.CreateDynamicFontFromOSFont(
25068             new[] { "PingFang SC", "Heiti SC", "Noto Sans CJK SC", "Arial Unicode MS" }, 28));
25069     }
25070     static void FinishFont(Font f)
25071     {
25072         _sysFont = f;
25073         Action<Font> wait = _sysFontWait;
25074         _sysFontWait = null;
25075         wait?.Invoke(f);
25076     }
25077     public static void KeepRuntime()
25078     {
25079         var wx = WxType();
25080         wx?.GetMethod("GetSystemInfoSync", Type.EmptyTypes)?.Invoke(null, null);
25081     }
25082     public static void OverrideTouch(GameObject es)
25083     {
25084         var t = Type.GetType("WXTouchInputOverride, Wx") ?? Type.GetType("WXTouchInputOverrid...
25085         if (t == null || es.GetComponent(t) != null) return;
25086         es.AddComponent(t);
25087     }
25088     // 编辑器，或微信开发者工具的模拟器。真机（ios / android / ohos / windows）一律 false。
25089     // 问不到平台就当真机，GM 入口不会出现。
25090     public static bool IsSimulator
25091     {
25092         get
25093         {
25094             if (Application.isEditor) return true;
25095             try
25096             {
25097                 object raw = Member(SystemInfo(), "platform");
25098                 string name = raw as string;
25099                 return name != null && name.Equals("devtools", StringComparison.OrdinalIgnore...
25100             }
25101             catch (Exception)
25102             {

```

```

25103         return false;
25104     }
25105 }
25106 }
25107 public static bool IsMiniGame
25108 {
25109     get
25110     {
25111 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
25112         return true;
25113 #else
25114         string p = Application.platform.ToString();
25115         return p.IndexOf("MiniGame", StringComparison.OrdinalIgnoreCase) >= 0
25116             || p.IndexOf("Weixin", StringComparison.OrdinalIgnoreCase) >= 0;
25117 #endif
25118     }
25119 }
25120 // 右上角那个胶囊按钮（「· · ·」+「○」）是微信画在游戏画布之上的，
25121 // 既不在 Screen.safeArea 里，Unity 也看不见它 -- 顶栏那排数值就这么被压住了。
25122 // 返回胶囊下沿占屏幕高度的比例（0~1），拿不到返回 -1。
25123 // 每帧都会问（ScreenFit.Apply 在 Update 里），所以只算一次；胶囊不会动。
25124 static float _capsule = float.NaN;
25125 public static float CapsuleBottomFrac()
25126 {
25127     if (!float.IsNaN(_capsule)) return _capsule;
25128     // 拿不到接口时的兜底：状态栏 + 胶囊在常见竖屏机型上大约占屏高的 9.5%。
25129     // 只在真机上兜底 -- 编辑器里没有胶囊，白让出一条会看不出真实排版。
25130     _capsule = IsMiniGame ? 0.095f : -1f;
25131     try
25132     {
25133         Type wx = WxType();
25134         object rect = wx?.GetMethod("GetMenuButtonBoundingClientRect", Type.EmptyTypes)
25135             ?.Invoke(null, null);
25136         float bottom = Num(rect, "bottom");
25137         float screenH = Num(SystemInfo(), "screenHeight");
25138         if (bottom > 0f && screenH > 0f) _capsule = bottom / screenH;
25139     }
25140     catch (Exception)
25141     {
25142         // 老版本基础库没有这个接口，兜底值已经设好了
25143     }
25144     return _capsule;
25145 }
25146 // 微信自己报的安全区，单位和 screenHeight 一致。团结导出的包里
25147 // Screen.safeArea 不一定填得对（刘海机上会退化成全屏），所以两边取大的那个。
25148 // 返回上下内缩占屏高的比例；拿不到就都是 -1。
25149 static Vector2 _inset = new Vector2(float.NaN, float.NaN);
25150 public static Vector2 SafeInsetFrac()
25151 {
25152     if (!float.IsNaN(_inset.x)) return _inset;

```

```

25153     _inset = new Vector2(-1f, -1f);
25154     try
25155     {
25156         object sys = SystemInfo();
25157         float screenH = Num(sys, "screenHeight");
25158         object area = Member(sys, "safeArea");
25159         float top = Num(area, "top");
25160         float bottom = Num(area, "bottom");
25161         if (screenH > 0f && bottom > top && bottom <= screenH)
25162             _inset = new Vector2(top / screenH, (screenH - bottom) / screenH);
25163     }
25164     catch (Exception)
25165     {
25166     }
25167     return _inset;
25168 }
25169 static object SystemInfo()
25170 {
25171     Type wx = WxType();
25172     return wx?.GetMethod("GetSystemInfoSync", Type.EmptyTypes)?.Invoke(null, null);
25173 }
25174 // 微信 SDK 里这些结构体有时是字段有时是属性，数值类型也不统一（float / double / int），
25175 // 所以统一按名字取再 Convert，别按具体类型强转。
25176 static object Member(object o, string name)
25177 {
25178     if (o == null) return null;
25179     Type t = o.GetType();
25180     var f = t.GetField(name);
25181     if (f != null) return f.GetValue(o);
25182     var p = t.GetProperty(name);
25183     return p?.GetValue(o);
25184 }
25185 static float Num(object o, string name)
25186 {
25187     object v = Member(o, name);
25188     return v == null ? 0f : Convert.ToSingle(v);
25189 }
25190 }
25191 }
25192 // ===== FILE: Assets/Scripts/Platform/WxRuntimeKeep.cs =====
25193 using UnityEngine;
25194 namespace InkLine
25195 {
25196     public static class WxRuntimeKeep
25197     {
25198         [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterAssembliesLoaded)]
25199         static void Keep()
25200         {
25201             WxBridge.KeepRuntime();
25202         }
25203     }
25204 }

```

```
25203   }
25204 }
25205 // ===== FILE: cloudfunctions/blackrosa-api/index.js =====
25206 /**
25207  * 墨字防线后端（CloudBase 云函数 + HTTP 访问服务），照 huahua-api / caizhu-api 模板。
25208  *
25209  * 路由（POST JSON）：
25210  *   /login    wx code2session / anon, 签发 JWT
25211  *   /save/pull 拉当前用户存档
25212  *   /save/push 上传存档（按 key 合并，updatedAt + baseRemoteUpdatedAt 防旧档回写）
25213  *   /rank/submit 上报已通关关数（只升不降），顺带更新昵称头像
25214  *   /rank/list  通关榜前 N 名 + 自己的名次
25215  *   /gameclub/daily 解密 wx.getGameClubData，回当天游戏圈发帖数
25216  *   /health    健康检查
25217  *
25218  * 环境变量：
25219  *   GAME_KEY=blackrosa
25220  *   BLACKROSA_JWT_SECRET      必填
25221  *   BLACKROSA_WX_APPID / BLACKROSA_WX_SECRET 微信 code2session
25222  *   BLACKROSA_SAVE_MAX_BYTES / BLACKROSA_TOKEN_TTL_SEC 可选
25223  *   BLACKROSA_RANK_MAX_CLEARED（默认 72）/ BLACKROSA_RANK_BLOCKED_UIDS（逗号分隔） 可选
25224  *
25225  * 集合：blackrosa_playerData（userId 唯一索引）
25226  *   blackrosa_rankings（userId 唯一索引；cleared desc + reachedAt asc）
25227  *   blackrosa_wxSessions（userId 唯一索引）
25228  */
25229 const { handleLogin } = require('./lib/auth');
25230 const { handlePull, handlePush } = require('./lib/save');
25231 const { handleSubmit, handleList } = require('./lib/rank');
25232 const { handleDailyPost } = require('./lib/game-club');
25233 const { respond, parseEvent, preflight } = require('./lib/http');
25234 const ROUTES = {
25235   'GET /health': async () => ({ ok: true, ts: Date.now() }),
25236   'POST /health': async () => ({ ok: true, ts: Date.now() }),
25237   'POST /login': handleLogin,
25238   'POST /save/pull': handlePull,
25239   'POST /save/push': handlePush,
25240   'POST /rank/submit': handleSubmit,
25241   'POST /rank/list': handleList,
25242   'POST /gameclub/daily': handleDailyPost,
25243 };
25244 exports.main = async (event, context) => {
25245   try {
25246     if (event && event.httpMethod === 'OPTIONS') {
25247       return preflight();
25248     }
25249     const req = parseEvent(event);
25250     const key = `${req.method} ${req.path}`;
25251     const handler = ROUTES[key];
25252     if (!handler) {
```

```
25253     return respond(404, { ok: false, code: 'NOT_FOUND', error: `no route: ${key}` });
25254   }
25255   const result = await handler(req, context);
25256   if (result && typeof result === 'object' && 'statusCode' in result) {
25257     return result;
25258   }
25259   return respond(200, { ok: true, data: result });
25260 } catch (error) {
25261   const code = error && error.code ? error.code : 'INTERNAL';
25262   const status = error && error.status ? error.status : 500;
25263   const message = (error && error.message) || String(error);
25264   console.error('[blackrosa-api] error:', code, message, error && error.stack);
25265   const out = { ok: false, code, error: message };
25266   if (error && error.data !== undefined) {
25267     out.data = error.data;
25268   }
25269   return respond(status, out);
25270 }
25271 };
25272 // ===== FILE: cloudfunctions/blackrosa-api/lib/auth.js =====
25273 const jwt = require('jsonwebtoken');
25274 const { httpError } = require('./http');
25275 const {
25276   getGameKey,
25277   gameKeyUpper,
25278   getJwtSecret: readJwtSecret,
25279   getTtlSec,
25280   getPlatformCredential,
25281 } = require('./config');
25282 const SUPPORTED_PLATFORMS = new Set(['wx', 'dy', 'tap', 'anon']);
25283 function getJwtSecret() {
25284   const secret = readJwtSecret();
25285   if (!secret) {
25286     throw httpError(500, 'NO_JWT_SECRET', `${gameKeyUpper()}_JWT_SECRET 未配置`);
25287   }
25288   return secret;
25289 }
25290 async function handleLogin(req) {
25291   const body = req.body || {};
25292   const platform = String(body.platform || '').toLowerCase();
25293   if (!SUPPORTED_PLATFORMS.has(platform)) {
25294     throw httpError(400, 'BAD_PLATFORM', `unsupported platform: ${platform}`);
25295   }
25296   let platformUid = '';
25297   let sessionKey = '';
25298   if (platform === 'wx') {
25299     const wx = await wxCode2Openid(body.code);
25300     platformUid = wx.openid;
25301     sessionKey = wx.sessionKey;
25302   } else if (platform === 'dy') {
```

```
25303   platformUid = await ttCode2Openid(body.code);
25304 } else if (platform === 'tap') {
25305   const id = String(body.taptapUserId || '').trim();
25306   if (!id) throw httpError(400, 'NO_TAP_ID', 'taptapUserId 缺失');
25307   platformUid = id;
25308 } else if (platform === 'anon') {
25309   const id = String(body.anonId || '').trim();
25310   if (!id) throw httpError(400, 'NO_ANON_ID', 'anonId 缺失');
25311   if (!/^[A-Za-z0-9_\-]{8,128}$/.test(id)) {
25312     throw httpError(400, 'BAD_ANON_ID', 'anonId 非法');
25313   }
25314   platformUid = id;
25315 }
25316 const userId = `${platform}:${platformUid}`;
25317 if (sessionKey) {
25318   // 游戏圈数据是用 session_key 加密的，解密接口要用到。存失败不影响登录。
25319   try {
25320     await require('./game-club').upsertWxSession(userId, sessionKey);
25321   } catch (error) {
25322     console.warn('[auth] save wx session failed', error && error.message);
25323   }
25324 }
25325 const ttlSec = getTtlSec();
25326 const gameKey = getGameKey();
25327 const now = Math.floor(Date.now() / 1000);
25328 const token = jwt.sign(
25329   { sub: userId, plt: platform, gk: gameKey, iat: now },
25330   getJwtSecret(),
25331   { expiresIn: ttlSec },
25332 );
25333 return {
25334   token,
25335   userId,
25336   platform,
25337   gameKey,
25338   expiresAt: (now + ttlSec) * 1000,
25339   ttlSec,
25340 };
25341 }
25342 function requireUser(req) {
25343   const authHeader = (req.headers && req.headers.authorization) || '';
25344   const match = /^Bearer\s+(.+)$/i.exec(authHeader);
25345   if (!match) throw httpError(401, 'NO_TOKEN', '缺少 Authorization: Bearer <token>');
25346   let payload;
25347   try {
25348     payload = jwt.verify(match[1].trim(), getJwtSecret());
25349   } catch (error) {
25350     throw httpError(401, 'BAD_TOKEN', error && error.message ? error.message : 'token 无效');
25351   }
25352   const userId = payload && payload.sub;
```

```
25353 if (userId || typeof userId !== 'string' || !userId.includes(':')) {
25354   throw httpError(401, 'BAD_TOKEN', 'token sub 非法');
25355 }
25356 const currentGameKey = getGameKey();
25357 if (payload.gk && payload.gk !== currentGameKey) {
25358   throw httpError(401, 'BAD_TOKEN', `token gameKey=${payload.gk} 与当前 GAME_KEY=${currentGameKey}...
25359 }
25360 return { userId, platform: payload.plt || userId.split(':')[0] };
25361 }
25362 async function wxCode2Openid(code) {
25363   const appid = getPlatformCredential('wx', 'APPID');
25364   const secret = getPlatformCredential('wx', 'SECRET');
25365   if (!appid || !secret) throw httpError(500, 'NO_WX_CFG', `${gameKeyUpper()}_WX_APPID/${gameKeyU...
25366   if (!code) throw httpError(400, 'NO_CODE', 'wx code 缺失');
25367   const url = `https://api.weixin.qq.com/sns/jscode2session?appid=${encodeURIComponent(appid)}&se...
25368   const data = await httpGetJson(url);
25369   if (!data || !data.openid) {
25370     throw httpError(401, 'WX_LOGIN_FAIL', `wx code2session 失败: ${JSON.stringify(data || {})});
25371   }
25372   return { openid: data.openid, sessionKey: data.session_key || '' };
25373 }
25374 async function ttCode2Openid(code) {
25375   const appid = getPlatformCredential('tt', 'APPID');
25376   const secret = getPlatformCredential('tt', 'SECRET');
25377   if (!appid || !secret) throw httpError(500, 'NO_TT_CFG', `${gameKeyUpper()}_TT_APPID/${gameKeyU...
25378   if (!code) throw httpError(400, 'NO_CODE', 'dy code 缺失');
25379   const data = await httpPostJson('https://developer.toutiao.com/api/apps/v2/jscode2session', { a...
25380   if (!data || data.err_no !== 0 || !data.data || !data.data.openid) {
25381     throw httpError(401, 'TT_LOGIN_FAIL', `dy code2session 失败: ${JSON.stringify(data || {})});
25382   }
25383   return data.data.openid;
25384 }
25385 function httpGetJson(url) {
25386   return httpRequestJson(url, 'GET');
25387 }
25388 function httpPostJson(url, body) {
25389   return httpRequestJson(url, 'POST', body);
25390 }
25391 function httpRequestJson(url, method, body) {
25392   if (typeof fetch !== 'function') {
25393     return Promise.reject(httpError(500, 'NO_FETCH', '当前 Node 运行时不支持 fetch, 请使用 Node 18+'));
25394   }
25395   const opts = { method, headers: { 'Content-Type': 'application/json' } };
25396   if (body !== undefined) {
25397     opts.body = JSON.stringify(body);
25398   }
25399   return fetch(url, opts).then(async (res) => {
25400     const text = await res.text();
25401     try {
25402       return JSON.parse(text);
```

```
25403     } catch (_) {
25404         return { _raw: text };
25405     }
25406 });
25407 }
25408 module.exports = {
25409     handleLogin,
25410     requireUser,
25411 };
25412 // ===== FILE: cloudfunctions/blackrosa-api/lib/config.js =====
25413 const DEFAULT_GAME_KEY = 'blackrosa';
25414 const DEFAULT_TTL_SEC = 7 * 24 * 3600;
25415 const DEFAULT_MAX_BYTES = 256 * 1024;
25416 function getGameKey() {
25417     const v = String(process.env.GAME_KEY || '').trim().toLowerCase();
25418     if (!v) return DEFAULT_GAME_KEY;
25419     if (!/^[a-z][a-z0-9_]{0,31}$/.test(v)) {
25420         throw new Error(`非法 GAME_KEY: ${v}`);
25421     }
25422     return v;
25423 }
25424 function gameKeyUpper() {
25425     return getGameKey().toUpperCase().replace(/[^A-Z0-9]/g, '_');
25426 }
25427 function readEnvPrefer(...keys) {
25428     for (const key of keys) {
25429         const value = process.env[key];
25430         if (value !== undefined && value !== null && String(value).length > 0) {
25431             return String(value);
25432         }
25433     }
25434     return '';
25435 }
25436 function getCollectionName(suffix) {
25437     const overrideKey = `${gameKeyUpper()}_${suffix.toUpperCase()}_COLLECTION`;
25438     const override = process.env[overrideKey];
25439     if (override) {
25440         return String(override);
25441     }
25442     return `${getGameKey()}_${suffix}`;
25443 }
25444 function getJwtSecret() {
25445     return readEnvPrefer(`${gameKeyUpper()}_JWT_SECRET`);
25446 }
25447 function getTtlSec() {
25448     const raw = readEnvPrefer(`${gameKeyUpper()}_TOKEN_TTL_SEC`);
25449     const v = Number(raw);
25450     return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_TTL_SEC;
25451 }
25452 function getMaxBytes() {
```

```
25453 const raw = readEnvPrefer(`${gameKeyUpper()}_SAVE_MAX_BYTES`);
25454 const v = Number(raw);
25455 return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_MAX_BYTES;
25456 }
25457 function getPlatformCredential(platform, field) {
25458   const upper = gameKeyUpper();
25459   const platformUpper = platform.toUpperCase();
25460   return readEnvPrefer(`${upper}_${platformUpper}_${field}`);
25461 }
25462 module.exports = {
25463   getGameKey,
25464   gameKeyUpper,
25465   readEnvPrefer,
25466   getCollectionName,
25467   getJwtSecret,
25468   getTtlSec,
25469   getMaxBytes,
25470   getPlatformCredential,
25471 };
25472 // ===== FILE: cloudfunctions/blackrosa-api/lib/db.js =====
25473 const tcb = require('@cloudbase/node-sdk');
25474 const { getCollectionName } = require('./config');
25475 let app = null;
25476 function getApp() {
25477   if (app) return app;
25478   app = tcb.init({
25479     env: process.env.TCB_ENV || tcb.SYMBOL_CURRENT_ENV,
25480   });
25481   return app;
25482 }
25483 function getDb() {
25484   return getApp().database();
25485 }
25486 function getCollection() {
25487   return getDb().collection(getCollectionName('playerData'));
25488 }
25489 function getRankingCollection() {
25490   return getDb().collection(getCollectionName('rankings'));
25491 }
25492 module.exports = {
25493   getApp,
25494   getDb,
25495   getCollection,
25496   getRankingCollection,
25497 };
25498 // ===== FILE: cloudfunctions/blackrosa-api/lib/game-club.js =====
25499 /**
25500  * 游戏圈每日发帖任务，照 hotpot-api 的 game-club.js。
25501  *
25502  * 客户端 wx.getGameClubData 拿到的是用 session_key 加密的数据，
```

```
25503 * 登录时把 session_key 存进 wxSessions, 这里解密后只回当天发帖数。
25504 * 奖励在客户端发 (跟着存档走), 服务端只负责证明「今天确实发过」。
25505 *
25506 * 集合: wxSessions (userId 唯一索引)
25507 */
25508 const crypto = require('crypto');
25509 const { httpError } = require('./http');
25510 const { getDb } = require('./db');
25511 const { getCollectionName } = require('./config');
25512 // wx.getGameClubData 的 dataType: 6 = 当天在本游戏圈发表的动态数。
25513 const DAILY_POST_TYPE = 6;
25514 function getSessionCollection() {
25515   return getDb().collection(getCollectionName('wxSessions'));
25516 }
25517 async function upsertWxSession(userId, sessionKey) {
25518   if (!sessionKey) return;
25519   const col = getSessionCollection();
25520   const now = Date.now();
25521   const res = await col.where({ userId }).limit(1).get();
25522   const existing = (res && Array.isArray(res.data) && res.data[0]) || null;
25523   if (existing && existing._id) {
25524     await col.doc(existing._id).update({ sessionKey, updatedAt: now });
25525     return;
25526   }
25527   await col.add({ userId, sessionKey, updatedAt: now });
25528 }
25529 async function readWxSessionKey(userId) {
25530   const res = await getSessionCollection().where({ userId }).limit(1).get();
25531   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
25532   return doc && typeof doc.sessionKey === 'string' ? doc.sessionKey : '';
25533 }
25534 function decrypt(sessionKey, encryptedData, iv) {
25535   try {
25536     const decipher = crypto.createDecipheriv(
25537       'aes-128-cbc',
25538       Buffer.from(sessionKey, 'base64'),
25539       Buffer.from(iv, 'base64'),
25540     );
25541     decipher.setAutoPadding(true);
25542     const text = Buffer.concat([
25543       decipher.update(Buffer.from(encryptedData, 'base64')),
25544       decipher.final(),
25545     ]).toString('utf8');
25546     return JSON.parse(text);
25547   } catch (error) {
25548     throw httpError(400, 'DECRYPT_FAIL', (error && error.message) || '游戏圈数据解密失败');
25549   }
25550 }
25551 function typeOf(entry) {
25552   const t = entry && entry.dataType;
```

```
25553   return typeof t === 'number' ? t : Number((t && t.type) || 0);
25554 }
25555 async function handleDailyPost(req) {
25556   // auth.js 登录时会反过来调这里存 session, 所以懒加载避免循环依赖。
25557   const { requireUser } = require('./auth');
25558   const { userId, platform } = requireUser(req);
25559   if (platform !== 'wx') throw httpError(400, 'NOT_WX', '游戏圈只在微信里可用');
25560   const body = req.body || {};
25561   const encryptedData = String(body.encryptedData || '').trim();
25562   const iv = String(body.iv || '').trim();
25563   if (!encryptedData || !iv) throw httpError(400, 'BAD_GAME_CLUB_PAYLOAD', 'encryptedData / iv 缺失');
25564   const sessionKey = await readWxSessionKey(userId);
25565   if (!sessionKey) throw httpError(400, 'NO_WX_SESSION', '微信 session 未就绪, 请重新登录');
25566   const data = decrypt(sessionKey, encryptedData, iv);
25567   const list = Array.isArray(data && data.dataList) ? data.dataList : [];
25568   const item = list.find((entry) => typeof(entry) === DAILY_POST_TYPE);
25569   const n = Number(item && item.value);
25570   return { postCount: Number.isFinite(n) && n > 0 ? Math.floor(n) : 0 };
25571 }
25572 module.exports = {
25573   DAILY_POST_TYPE,
25574   upsertWxSession,
25575   handleDailyPost,
25576 };
25577 // ===== FILE: cloudfunctions/blackrosa-api/lib/http.js =====
25578 const { getGameKey } = require('./config');
25579 const CORS_HEADERS = {
25580   'Access-Control-Allow-Origin': '*',
25581   'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
25582   'Access-Control-Allow-Headers': 'Content-Type,Authorization',
25583   'Access-Control-Max-Age': '86400',
25584 };
25585 function respond(statusCode, body, extraHeaders = {}) {
25586   return {
25587     statusCode,
25588     headers: {
25589       'Content-Type': 'application/json; charset=utf-8',
25590       ...CORS_HEADERS,
25591       ...extraHeaders,
25592     },
25593     body: typeof body === 'string' ? body : JSON.stringify(body),
25594     isBase64Encoded: false,
25595   };
25596 }
25597 function preflight() {
25598   return {
25599     statusCode: 204,
25600     headers: { ...CORS_HEADERS },
25601     body: '',
25602     isBase64Encoded: false,
```

```
25603   };
25604 }
25605 function parseEvent(event) {
25606   event = event || {};
25607   if (event.httpMethod) {
25608     const method = String(event.httpMethod).toUpperCase();
25609     let rawBody = event.body || "";
25610     if (event.isBase64Encoded && rawBody) {
25611       try {
25612         rawBody = Buffer.from(rawBody, 'base64').toString('utf8');
25613       } catch (_) {}
25614     }
25615     let body = {};
25616     if (rawBody) {
25617       try {
25618         body = JSON.parse(rawBody);
25619       } catch (_) {
25620         body = {};
25621       }
25622     }
25623     return {
25624       method,
25625       path: normalizePath(event.path || '/'),
25626       body,
25627       headers: lowercaseHeaders(event.headers || {}),
25628       query: event.queryStringParameters || {},
25629       raw: event,
25630     };
25631   }
25632   const action = String(event.action || "").replace(/^\//+, "");
25633   return {
25634     method: 'POST',
25635     path: action ? `/${action}` : '/',
25636     body: event.body || {},
25637     headers: lowercaseHeaders(event.headers || {}),
25638     query: {},
25639     raw: event,
25640   };
25641 }
25642 function normalizePath(path) {
25643   if (!path) return '/';
25644   let p = String(path);
25645   if (!p.startsWith('/')) p = `/${p}`;
25646   const apiPrefix = `${getGameKey()}-api`;
25647   p = p.replace(new RegExp(`^(?:${escapeRegExp(apiPrefix)})(?=/|$)`), "");
25648   if (p === "") p = '/';
25649   if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1);
25650   return p;
25651 }
25652 function lowercaseHeaders(headers) {
```

```
25653   const out = {};
25654   for (const key of Object.keys(headers || {})) {
25655     out[key.toLowerCase()] = headers[key];
25656   }
25657   return out;
25658 }
25659 function httpError(status, code, message) {
25660   const err = new Error(message || code);
25661   err.status = status;
25662   err.code = code;
25663   return err;
25664 }
25665 function escapeRegExp(text) {
25666   return String(text).replace(/[\.*+?^${}()|[\]\]/g, '\\$&');
25667 }
25668 module.exports = {
25669   respond,
25670   preflight,
25671   parseEvent,
25672   httpError,
25673 };
25674 // ===== FILE: cloudfunctions/blackrosa-api/lib/rank.js =====
25675 /**
25676  * 通关排行榜，照 hotpot-api 的 rank.js。只有一张榜：已通关关数。
25677  *
25678  * 数据模型（collection: rankings，每人一条）：
25679  * {
25680  *   _id,
25681  *   userId, platform,
25682  *   cleared: number,    // 已通关关数
25683  *   reachedAt: number (ms), // 第一次到达这个关数的时间，同分先到者排前面
25684  *   displayName, avatarUrl,
25685  *   createdAt, updatedAt
25686  * }
25687  *
25688  * 索引：userId 唯一；cleared desc + reachedAt asc。
25689  */
25690 const crypto = require('crypto');
25691 const { requireUser } = require('./auth');
25692 const { getDb, getRankingCollection } = require('./db');
25693 const { httpError } = require('./http');
25694 const { readEnvPrefer, gameKeyUpper } = require('./config');
25695 const DEFAULT_MAX_CLEARED = 72;
25696 const DEFAULT_LIST_LIMIT = 50;
25697 const MAX_LIST_LIMIT = 100;
25698 function envInt(suffix, fallback) {
25699   const n = Number(readEnvPrefer(`${gameKeyUpper()}_${suffix}`));
25700   return Number.isFinite(n) && n > 0 ? Math.floor(n) : fallback;
25701 }
25702 // GM / 测试账号：逗号分隔的 userId，不写榜，免得一键全通关的进度挂在榜首。
```

```
25703 function blockedUsers() {
25704   return new Set(
25705     readEnvPrefer(`${gameKeyUpper()}_RANK_BLOCKED_UIDS`)
25706     .split(',')
25707     .map((s) => s.trim())
25708     .filter(Boolean),
25709   );
25710 }
25711 function normalizeCleared(value) {
25712   const n = Number(value);
25713   if (!Number.isFinite(n) || n < 0) {
25714     throw httpError(400, 'BAD_CLEARED', 'cleared 必须为非负整数');
25715   }
25716   const cleared = Math.floor(n);
25717   const max = envInt('RANK_MAX_CLEARED', DEFAULT_MAX_CLEARED);
25718   if (cleared > max) {
25719     throw httpError(400, 'CLEARED_TOO_HIGH', `cleared 超出上限: ${cleared} > ${max}`);
25720   }
25721   return cleared;
25722 }
25723 function normalizeLimit(value) {
25724   const n = Number(value);
25725   if (!Number.isFinite(n) || n <= 0) return DEFAULT_LIST_LIMIT;
25726   return Math.min(Math.floor(n), MAX_LIST_LIMIT);
25727 }
25728 function displayNameForUser(userId) {
25729   const digest = crypto.createHash('sha1').update(String(userId)).digest('hex');
25730   const suffix = String(parseInt(digest.slice(0, 8), 16) % 10000).padStart(4, '0');
25731   return `墨客${suffix}`;
25732 }
25733 function sanitizeDisplayName(value, fallback) {
25734   const text = String(value || '').trim().replace(/[\r\n\t]/g, ' ').slice(0, 16);
25735   // 微信拒绝授权时会给一个统一的「微信用户」，当没给。
25736   if (!text || text === '微信用户') return fallback;
25737   return text;
25738 }
25739 function sanitizeAvatarUrl(value, fallback) {
25740   const text = String(value || '').trim();
25741   if (!text || text.length > 1024 || !/^https?:\/\//i.test(text)) return fallback || '';
25742   return text;
25743 }
25744 function publicRecord(doc, userId, rank) {
25745   if (!doc) return null;
25746   return {
25747     rank: rank || 0,
25748     cleared: Number(doc.cleared) || 0,
25749     displayName: doc.displayName || displayNameForUser(doc.userId || ''),
25750     avatarUrl: doc.avatarUrl || '',
25751     isMe: !!userId && doc.userId === userId,
25752   };
}
```

```
25753 }
25754 // 编辑器用 anon 身份联调。微信玩家看不到 anon 记录，编辑器里能看到全部，
25755 // 这样本地调界面有数据可看，线上榜也不会混进测试号。
25756 function audience(platform) {
25757   return platform === 'anon' ? {} : { platform: getDb().command.neq('anon') };
25758 }
25759 async function findMine(col, userId, platform) {
25760   const res = await col.where({ userId }).limit(1).get();
25761   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
25762   if (!doc || !(Number(doc.cleared) > 0)) return null;
25763   const _ = getDb().command;
25764   const cleared = Number(doc.cleared);
25765   const reachedAt = Number(doc.reachedAt) || 0;
25766   const scope = audience(platform);
25767   const [ahead, tied] = await Promise.all([
25768     col.where({ ...scope, cleared: _.gt(cleared) }).count(),
25769     col.where({ ...scope, cleared, reachedAt: _.lt(reachedAt) }).count(),
25770   ]);
25771   const rank = (Number(ahead && ahead.total) || 0) + (Number(tied && tied.total) || 0) + 1;
25772   return { rank, doc };
25773 }
25774 async function handleSubmit(req) {
25775   const { userId, platform } = requireUser(req);
25776   const body = req.body || {};
25777   const cleared = normalizeCleared(body.cleared);
25778   if (blockedUsers().has(userId)) {
25779     return { updated: false, reason: 'BLOCKED', record: null };
25780   }
25781   const col = getRankingCollection();
25782   const res = await col.where({ userId }).limit(1).get();
25783   const existing = (res && Array.isArray(res.data) && res.data[0]) || null;
25784   const now = Date.now();
25785   const displayName = sanitizeDisplayName(
25786     body.displayName,
25787     (existing && existing.displayName) || displayNameForUser(userId),
25788   );
25789   const avatarUrl = sanitizeAvatarUrl(body.avatarUrl, (existing && existing.avatarUrl) || '');
25790   if (!existing) {
25791     if (cleared <= 0) return { updated: false, reason: 'NO_PROGRESS', record: null };
25792     const docData = {
25793       userId,
25794       platform,
25795       cleared,
25796       reachedAt: now,
25797       displayName,
25798       avatarUrl,
25799       createdAt: now,
25800       updatedAt: now,
25801     };
25802     await col.add(docData);
```

```

25803   return { updated: true, mode: 'insert', record: publicRecord(docData, userId, 0) };
25804 }
25805 const prev = Number(existing.cleared) || 0;
25806 const patch = {};
25807 // 只升不降：清档重玩、换设备拿到旧档都不会把榜上的成绩拉下来。
25808 if (cleared > prev) {
25809   patch.cleared = cleared;
25810   patch.reachedAt = now;
25811 }
25812 if (existing.displayName !== displayName) patch.displayName = displayName;
25813 if ((existing.avatarUrl || '') !== avatarUrl) patch.avatarUrl = avatarUrl;
25814 if (Object.keys(patch).length === 0) {
25815   return { updated: false, reason: 'NOT_BETTER', record: publicRecord(existing, userId, 0) };
25816 }
25817 patch.updatedAt = now;
25818 await col.doc(existing._id).update(patch);
25819 return {
25820   updated: true,
25821   mode: patch.cleared !== undefined ? 'update' : 'profile_update',
25822   record: publicRecord({ ...existing, ...patch }, userId, 0),
25823 };
25824 }
25825 async function handleList(req) {
25826   const { userId, platform } = requireUser(req);
25827   const limit = normalizeLimit((req.body || {}).limit);
25828   const col = getRankingCollection();
25829   const res = await col
25830     .where({ ...audience(platform), cleared: getDb().command.gt(0) })
25831     .orderBy('cleared', 'desc')
25832     .orderBy('reachedAt', 'asc')
25833     .limit(limit)
25834     .get();
25835   const rows = res && Array.isArray(res.data) ? res.data : [];
25836   const list = rows.map((doc, i) => publicRecord(doc, userId, i + 1));
25837   const mine = await findMine(col, userId, platform);
25838   return { list, mine: mine ? publicRecord(mine.doc, userId, mine.rank) : null };
25839 }
25840 module.exports = {
25841   handleSubmit,
25842   handleList,
25843 };
25844 // ===== FILE: cloudfunctions/blackrosa-api/lib/save.js =====
25845 /**
25846  * 存档 pull / push 处理
25847  *
25848  * 数据模型 (MongoDB collection: playerData) :
25849  * {
25850  *   _id,
25851  *   userId: 'wx:...' | 'dy:...' | 'tap:...' | 'anon:...',
25852  *   schemaVersion: number,

```

```
25853 *   updatedAt: number (ms),
25854 *   baseRemoteUpdatedAt: number (ms), // 客户端本地修改基于的云端版本
25855 *   clientFingerprint: string,
25856 *   payload: { [key: string]: string }, // 客户端上行的白名单 key -> 原始JSON 字符串
25857 *   payloadKeys: string[],
25858 *   lastWriteAt: number (ms, 服务端写入时间, 用于排障)
25859 * }
25860 */
25861 const { httpError } = require('./http');
25862 const { requireUser } = require('./auth');
25863 const { getCollection } = require('./db');
25864 const { getMaxBytes } = require('./config');
25865 async function handlePull(req) {
25866   const { userId, platform } = requireUser(req);
25867   const col = getCollection();
25868   const res = await col.where({ userId }).limit(1).get();
25869   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
25870   if (!doc) {
25871     return {
25872       userId,
25873       platform,
25874       exists: false,
25875       schemaVersion: 0,
25876       updatedAt: 0,
25877       payload: {},
25878       payloadKeys: [],
25879     };
25880   }
25881   return {
25882     userId,
25883     platform,
25884     exists: true,
25885     schemaVersion: doc.schemaVersion || 0,
25886     updatedAt: doc.updatedAt || 0,
25887     payload: doc.payload || {},
25888     payloadKeys: Array.isArray(doc.payloadKeys)
25889       ? doc.payloadKeys
25890       : Object.keys(doc.payload || {}),
25891     clientFingerprint: doc.clientFingerprint || "",
25892   };
25893 }
25894 async function handlePush(req) {
25895   const { userId, platform } = requireUser(req);
25896   const body = req.body || {};
25897   const schemaVersion = Number(body.schemaVersion);
25898   const updatedAt = Number(body.updatedAt);
25899   const baseRemoteUpdatedAt = Number(body.baseRemoteUpdatedAt || 0);
25900   const payload = body.payload;
25901   const clientFingerprint = String(body.clientFingerprint || "").slice(0, 200);
25902   const force = body.force === true;
```

```
25903 if (!Number.isFinite(schemaVersion) || schemaVersion <= 0) {
25904   throw httpError(400, 'BAD_SCHEMA', 'schemaVersion 非法');
25905 }
25906 if (!Number.isFinite(updatedAt) || updatedAt <= 0) {
25907   throw httpError(400, 'BAD_UPDATED_AT', 'updatedAt 非法');
25908 }
25909 if (!Number.isFinite(baseRemoteUpdatedAt) || baseRemoteUpdatedAt < 0) {
25910   throw httpError(400, 'BAD_BASE_REMOTE_UPDATED_AT', 'baseRemoteUpdatedAt 非法');
25911 }
25912 if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
25913   throw httpError(400, 'BAD_PAYLOAD', 'payload 必须是 object');
25914 }
25915 // 体积上限 (粗略按 JSON 序列化后字节数)
25916 const size = Buffer.byteLength(JSON.stringify(payload), 'utf8');
25917 const maxBytes = getMaxBytes();
25918 if (size > maxBytes) {
25919   throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${size}B > ${maxBytes}B`);
25920 }
25921 // 每个 value 必须是字符串 (与客户端 PersistService 约定一致)
25922 const payloadKeys = [];
25923 for (const k of Object.keys(payload)) {
25924   const v = payload[k];
25925   if (typeof v !== 'string') {
25926     throw httpError(400, 'BAD_PAYLOAD_VALUE', `payload[${k}] 必须是字符串`);
25927   }
25928   payloadKeys.push(k);
25929 }
25930 const col = getCollection();
25931 const existingRes = await col.where({ userId }).limit(1).get();
25932 const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
25933 if (existing && !force) {
25934   const prevUpdatedAt = Number(existing.updatedAt) || 0;
25935   if (updatedAt < prevUpdatedAt || baseRemoteUpdatedAt < prevUpdatedAt) {
25936     // 服务端已有更新的版本 -- 拒绝, 并把远端版本回传让客户端合并
25937     throw Object.assign(
25938       httpError(
25939         409,
25940         'STALE_UPDATE',
25941         `服务端已有更新版本 remote=${prevUpdatedAt} > local=${updatedAt}, base=${baseRemoteUpdatedAt}`,
25942       ),
25943       {
25944         data: {
25945           remote: {
25946             schemaVersion: existing.schemaVersion || 0,
25947             updatedAt: prevUpdatedAt,
25948             payload: existing.payload || {},
25949             payloadKeys: Array.isArray(existing.payloadKeys)
25950               ? existing.payloadKeys
25951               : Object.keys(existing.payload || {}),
25952           },

```

```
25953     },
25954     },
25955   );
25956   }
25957 }
25958 // 上行只覆盖本次出现的 key，缺 key 不删云端已有的那份。
25959 let mergedPayload = { ...(existing && existing.payload ? existing.payload : {}) };
25960 for (const k of Object.keys(payload)) {
25961   const v = payload[k];
25962   if (v === undefined || v === null) {
25963     delete mergedPayload[k];
25964   } else {
25965     mergedPayload[k] = v;
25966   }
25967 }
25968 const mergedPayloadKeys = Object.keys(mergedPayload);
25969 const mergedSize = Buffer.byteLength(JSON.stringify(mergedPayload), 'utf8');
25970 if (mergedSize > maxBytes) {
25971   throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${mergedSize}B > ${maxBytes}B`);
25972 }
25973 const now = Date.now();
25974 const docData = {
25975   userId,
25976   platform,
25977   schemaVersion,
25978   updatedAt,
25979   baseRemoteUpdatedAt,
25980   clientFingerprint,
25981   payload: mergedPayload,
25982   payloadKeys: mergedPayloadKeys,
25983   lastWriteAt: now,
25984 };
25985 if (existing && existing._id) {
25986   await col.doc(existing._id).update(docData);
25987   return { userId, updatedAt, savedAt: now, mode: 'update', sizeBytes: mergedSize };
25988 }
25989 const addRes = await col.add(docData);
25990 return {
25991   userId,
25992   updatedAt,
25993   savedAt: now,
25994   mode: 'insert',
25995   sizeBytes: size,
25996   _id: addRes && (addRes.id || (addRes._id)),
25997 };
25998 }
25999 module.exports = {
26000   handlePull,
26001   handlePush,
26002 };
```